

Laporan Tugas Besar 2
IF2211 Strategi Algoritma
Pemanfaatan Algoritma BFS dan DFS dalam Mekanisme
Penelusuran CSS pada Pohon *Document Object Model*



Kelompok “TimsesDewaPetir”

Anggota:

Nicholas Wise Saragih Sumbayak	(13524037)
Narendra Dharma Wistara M	(13524044)
Amanda Aurellia Salsabilla	(13524131)

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
BANDUNG
2025

PERNYATAAN

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.



Nicholas Wise Saragih Sumbayak



Narendra Dharma Wistara M



Amanda Aurellia Salsabilla

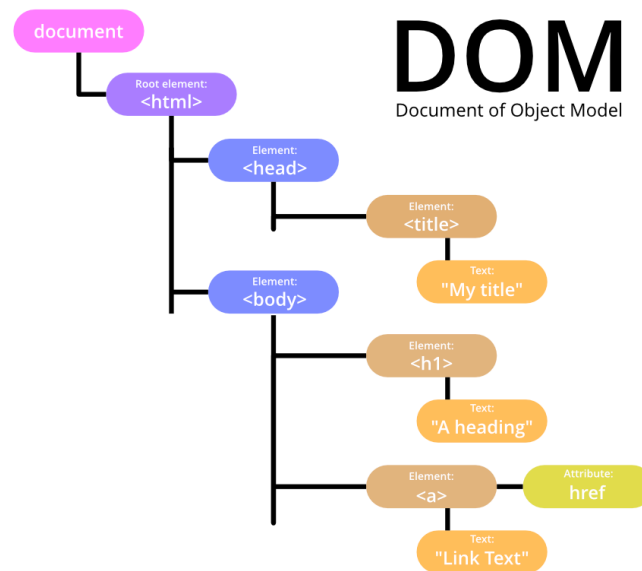
DAFTAR ISI

PERNYATAAN.....	2
DAFTAR ISI.....	3
BAB 1.....	5
BAB 2.....	9
2.1. Document Object Model (DOM).....	9
2.1.1. HTML.....	9
2.1.2. CSS.....	10
2.1.3. Web Scraping.....	11
2.2. Algoritma Traversal Graf.....	11
2.2.1. Depth-First Search.....	11
2.2.2. Breadth-First Search.....	12
2.2.3. Lowest Common Ancestor (LCA) melalui Binary Lifting.....	14
2.3. Dasar Teori Lainnya.....	15
2.3.1. Containerization menggunakan Docker.....	15
2.3.2. Virtual Machines melalui Microsoft Azure.....	16
2.3.3. Multithreading melalui Goroutines.....	16
BAB 3.....	18
3.1. Algoritma Breadth-First Search.....	18
3.1.1. Breadth-First Search secara Sekuensial.....	18
3.1.2. Breadth-First Search secara Konkuren.....	18
3.2. Algoritma Depth-First Search.....	19
3.2.1. Depth-First Search secara Sekuensial.....	19
3.2.2. Depth-First Search secara Konkuren.....	19
BAB 4.....	20
4.1. Implementasi.....	20
4.1.1. Tech Stack.....	20
4.1.1.1. Backend.....	20
4.1.1.2. Frontend.....	21
4.1.1.3. Infrastruktur dan Deployment.....	21
4.1.2. Struktur Proyek.....	21
4.1.3. Kode Program.....	22
4.1.3.1. Implementasi Pengambilan dan Parsing HTML (scraper.go).....	22
4.1.3.2. Representasi Pohon DOM (node.go).....	24
4.1.3.3. Algoritma Evaluasi Selektor (selector.go).....	25
4.1.3.4. Implementasi BFS dan DFS.....	30
4.1.3.5. Antarmuka Pengguna (Frontend).....	33
4.1.3.6. Implementasi Deployment dan Orkestrasi Kontainer.....	35
4.2. Pengujian.....	36

4.2.1. Test Case 1.....	36
4.2.2. Test Case 2.....	40
4.2.3. Test Case 3.....	43
4.2.4. Test Case 4.....	45
4.2.5. Test Case 5.....	49
4.3. Analisis Pengujian.....	49
BAB 5.....	51
5.1 Kesimpulan.....	51
5.2 Saran.....	51
LAMPIRAN.....	53
DAFTAR PUSTAKA.....	55

BAB 1

DESKRIPSI TUGAS



Gambar 1. Struktur Pohon DOM

<https://www.geeksforgeeks.org/java/what-is-document-object-in-java-dom/>

Document Object Model (DOM) merupakan representasi terstruktur dari dokumen HTML dalam bentuk *tree* yang memungkinkan setiap elemen pada halaman web diakses, dimodifikasi, dan dimanipulasi oleh program. Struktur ini merepresentasikan hubungan antar elemen HTML seperti parent, child, dan sibling sehingga memudahkan pengolahan dokumen secara terprogram.

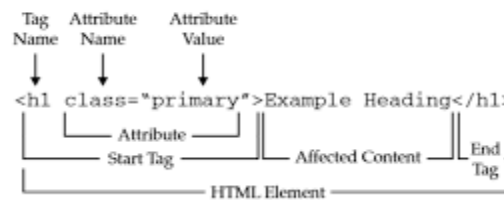
Struktur sebuah file HTML pada umumnya meliputi elemen root <html> yang berisi dua bagian utama yaitu <head> dan <body>. Bagian <head> berisi metadata seperti judul halaman, link stylesheet, dan script, sedangkan <body> berisi konten utama halaman seperti teks, gambar, tombol, dan elemen HTML lainnya yang ditampilkan kepada pengguna.

Sebuah website menampilkan sebuah DOM melalui sebuah file bernama Cascading Style Sheets (CSS) yang bertujuan mendeklarasikan visualisasi elemen-elemen pada pohon. Deklarasi visualisasi ditempelkan pada elemen-elemen yang berkaitan melalui CSS Selector.

Pada Tugas Besar 2 Strategi Algoritma ini, mahasiswa diminta untuk menelusuri dan mencari elemen pada struktur DOM berdasarkan CSS Selector tertentu dengan menggunakan algoritma penelusuran graf yaitu Breadth First Search (BFS) dan Depth First Search (DFS). Algoritma tersebut digunakan untuk menjelajahi struktur pohon DOM guna menemukan elemen yang sesuai dengan selector yang diberikan.

Komponen-komponen penting yang terdapat pada tugas besar ini antara lain:

1. Tags



Gambar 2. Struktur Syntax HTML

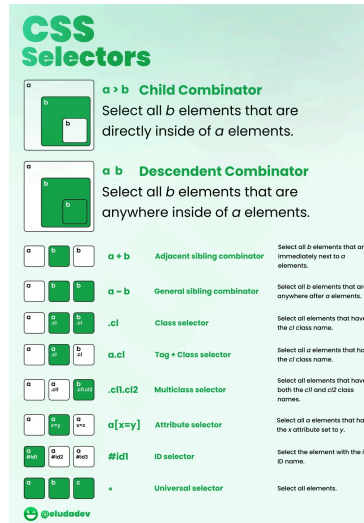
<https://tutorial.techaltum.com/htmlTags.html>

Tag pada HTML merupakan penanda (markup) yang digunakan untuk mendefinisikan struktur dan jenis elemen dalam sebuah dokumen HTML. Tag memberi tahu browser bagaimana suatu bagian konten harus ditafsirkan dan ditampilkan pada halaman web.

Secara umum, tag HTML ditulis menggunakan tanda kurung sudut (<>). Sebagian besar elemen HTML memiliki pasangan tag pembuka dan tag penutup. Tag pembuka menandai awal suatu elemen, sedangkan tag penutup menandai akhir elemen tersebut. Tag penutup ditulis dengan menambahkan tanda garis miring / sebelum nama tag. Terdapat beberapa jenis tag HTML yang *self-closing*.

Dalam struktur DOM, setiap tag HTML direpresentasikan sebagai node pada pohon DOM. Hubungan antar tag membentuk struktur hierarki seperti parent, child, dan sibling. Struktur inilah yang memungkinkan dokumen HTML diproses sebagai sebuah pohon ketika dilakukan penelusuran menggunakan algoritma seperti BFS atau DFS.

2. CSS Selector



Gambar 3. Visualisasi CSS Selector

https://www.reddit.com/r/webdev/comments/ur6v5m/css_selectors_visually_explained/

CSS Selector adalah mekanisme pada CSS untuk memilih elemen HTML tertentu pada struktur DOM agar dapat diberikan aturan style. Dengan selector, kita dapat menentukan elemen mana yang akan dikenai aturan visual seperti warna, ukuran, atau tata letak. Beberapa selector dasar yang umum digunakan adalah sebagai berikut.

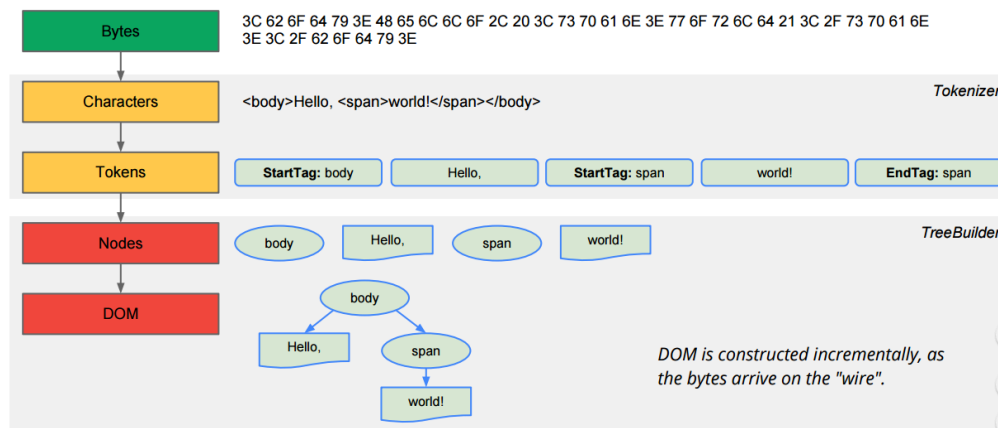
- **Tag Selector**
Memilih elemen berdasarkan nama tag HTML. Contoh: p memilih semua elemen <p>.
- **Class Selector**
Memilih elemen yang memiliki atribut class tertentu, ditulis dengan tanda titik (.). Contoh: .box
- **ID Selector**
Memilih elemen dengan atribut id tertentu, ditulis dengan tanda pagar (#). Contoh: #header
- **Universal Selector**
Memilih semua elemen HTML.
Contoh: *

Combinator merupakan cara untuk menggabungkan beberapa jenis selector dalam penentuan elemen yang akan terpengaruh, dan terdiri dari:

- **Child Combinator (>)**

- Descendent Combinator (“ “)
- Adjacent Sibling Combinator (+)
- General Sibling Combinator (~)

3. HTML Parsing



Gambar 4. Proses Parsing Pohon HTML

<https://stackoverflow.com/questions/3150293/how-does-a-parser-for-example-html-work>

HTML Parsing adalah proses membaca dokumen HTML dan mengubahnya menjadi struktur data pohon (tree) yang merepresentasikan hubungan antar elemen HTML. Pada proses ini, parser memproses dokumen HTML secara berurutan, mengenali tag pembuka dan tag penutup, kemudian menyusunnya ke dalam struktur hierarki.

Setiap tag HTML direpresentasikan sebagai node pada pohon. Tag yang berada di dalam tag lain akan menjadi child node, sedangkan tag yang membungkusnya menjadi parent node. Node paling atas pada struktur ini disebut root, yang umumnya merupakan elemen `<html>`. Dari node root tersebut, elemen-elemen lain seperti `<head>` dan `<body>` akan menjadi cabang yang membentuk struktur pohon DOM.

Hasil dari proses parsing adalah DOM Tree, yaitu representasi pohon dari dokumen HTML yang memungkinkan elemen-elemen pada halaman web ditelusuri, diakses, dan dimanipulasi secara terstruktur. Struktur pohon ini kemudian dapat digunakan dalam proses penelusuran elemen, misalnya dengan menggunakan algoritma Breadth First Search (BFS) atau Depth First Search (DFS).

BAB 2

LANDASAN TEORI

2.1. Document Object Model (DOM)

Document Object Model (DOM) adalah antarmuka pemrograman yang merepresentasikan struktur dokumen HTML sebagai sebuah pohon (tree) yang terdiri dari objek-objek terstruktur. Menurut Mozilla Developer Network (MDN): "*The DOM represents a document with a logical tree. Each branch of the tree ends in a node, and each node contains objects. DOM methods allow programmatic access to the tree. With them, you can change the document's structure, style, or content*". Dengan representasi ini, setiap elemen, atribut, dan teks dalam dokumen HTML menjadi sebuah node yang dapat diakses dan dimanipulasi secara terprogram.

Struktur pohon DOM memungkinkan hubungan antar elemen terdefinisi secara hierarkis. Simpul teratas yang seringkali disebut *root node* (simpul induk) umumnya adalah elemen `<html>` yang kemudian dibagi menjadi `<head>` atau `<body>`. Dari simpul tersebut, seluruh elemen lain membentuk cabang (*branch*) dan daun (*leaf*), sehingga membentuk struktur yang memungkinkan traversal secara sistematis menggunakan algoritma seperti BFS maupun DFS.

2.1.1. HTML

HTML adalah bahasa markup standar yang digunakan untuk membangun struktur halaman web. Menurut MDN Web Docs, HTML terdiri dari serangkaian elemen yang digunakan untuk membungkus atau menandai bagian konten tertentu agar dapat ditampilkan dengan cara tertentu oleh browser. Setiap elemen HTML diwakili oleh tag yang ditulis menggunakan tanda kurung sudut (`<>`).

Sebuah elemen HTML pada umumnya terdiri dari:

- Tag pembuka (opening tag): menandai awal elemen, misalnya `<p>`.
- Konten: isi dari elemen tersebut
- Tag penutup (closing tag): menandai akhir elemen, misalnya `</p>`.
- Atribut (opsional): properti tambahan yang ditulis di dalam tag pembuka, misalnya `class`, `id`, dan `href`.

Dalam konteks DOM, dokumen HTML yang telah di-parsing akan menghasilkan struktur pohon di mana setiap elemen HTML menjadi sebuah node. Hubungan antar elemen mencerminkan hubungan *parent-child* dan *sibling* yang merupakan dasar dari traversal pohon DOM.

2.1.2. CSS

Cascading Style Sheets (CSS) adalah bahasa yang digunakan untuk mendeskripsikan tampilan dokumen HTML. Untuk menghubungkan aturan style dengan elemen HTML yang dituju, CSS menggunakan mekanisme yang disebut CSS Selector. Menurut MDN Web Docs, CSS Selector mendefinisikan pola yang digunakan untuk mencocokkan elemen-elemen yang akan diberikan “aturan” gaya (*styling*). Terdapat beberapa jenis selector dasar, antara lain:

- Tag Selector: memilih elemen berdasarkan nama tag HTML.
Contoh: `p` memilih semua elemen `<p>`
- Class Selector: memilih elemen yang memiliki atribut class tertentu, ditandai dengan tanda titik (`.`).
Contoh: `.container`
- ID Selector: memilih elemen dengan atribut id tertentu, ditandai dengan tanda pagar (`#`).
Contoh: `#header`
- Universal Selector (`*`): memilih semua elemen HTML.
- Attribute Selector: memilih elemen berdasarkan atribut tertentu.
Contoh: `[type="text"]`

Selain selektor dasar, CSS juga mengenal combinator yang digunakan untuk menggabungkan selector berdasarkan hubungan antar elemen dalam pohon DOM:

- Descendant Combinator (spasi): memilih semua elemen B yang merupakan keturunan dari elemen A. Contoh: `div p`
- Child Combinator (`>`): memilih elemen B yang merupakan anak langsung dari elemen A.
Contoh: `ul > li`

- Adjacent Sibling Combinator (+): memilih elemen B yang secara langsung mengikuti elemen A dan memiliki *parent* yang sama.

Contoh: $h1 + p$

- General Sibling Combinator (~): memilih semua elemen B yang merupakan saudara (*sibling*) dari elemen A.

Contoh: $h1 \sim p$

2.1.3. Web Scraping

Web scraping adalah teknik pengambilan data dari sebuah halaman web secara otomatis dengan cara mengunduh konten HTML dari URL tertentu dan memprosesnya secara terprogram. Proses ini umumnya melibatkan dua tahap utama: (1) pengiriman HTTP request ke server untuk mendapatkan konten HTML mentah dan (2) parsing konten HTML tersebut menjadi struktur data yang dapat diolah lebih lanjut.

Dalam konteks proyek ini, *web scraping* dilakukan dengan mengirimkan HTTP GET request ke URL yang dimasukkan oleh pengguna, kemudian respons HTML yang diterima diproses oleh HTML parser untuk membangun pohon DOM. Proses parsing HTML bekerja dengan membaca dokumen secara sekuensial, mengenali token-token seperti tag pembuka, tag penutup, atribut, dan teks, lalu menyusunnya ke dalam struktur hierarki pohon DOM.

2.2. Algoritma Traversal Graf

Dalam dunia pemrograman dan informatika, seringkali sebuah persoalan digambarkan dalam bentuk graf yang akan dijelajahi untuk mencari solusi. Dalam konteks ini, traversal (penjelajahan) graf adalah proses mengunjungi setiap node dalam suatu struktur graf atau pohon secara sistematis. Dalam konteks pohon DOM, traversal digunakan untuk menelusuri seluruh elemen HTML guna menemukan elemen yang sesuai dengan CSS selector yang diberikan. Dua algoritma traversal yang digunakan dalam tugas besar ini adalah Depth-First Search (DFS) dan Breadth-First Search (BFS).

2.2.1. Depth-First Search

Depth-First Search (DFS) adalah algoritma traversal graf yang bekerja dengan cara menelusuri graf sedalam mungkin pada setiap cabang sebelum melakukan backtracking (mundur). Penelusuran DFS pada graf pada dasarnya serupa dengan penelusuran pada struktur pohon (tree), namun perbedaannya terletak pada fakta bahwa

graf dapat memiliki siklus (cycle), sehingga suatu node berpotensi dikunjungi berulang kali. Untuk menghindari pemrosesan node yang sama lebih dari sekali, penelusuran graf umumnya memanfaatkan penanda untuk melacak node yang sudah dieksplorasi, seperti penggunaan array boolean.

Pada pohon DOM, DFS dimulai dari node root (`<html>`) dan menelusuri setiap cabang secara mendalam terlebih dahulu sebelum berpindah ke cabang berikutnya. Algoritma ini umumnya diimplementasikan menggunakan rekursi atau struktur data *stack*.

Langkah-langkah algoritma DFS pada pohon DOM adalah sebagai berikut:

1. Mulai dari node root, tandai node tersebut sebagai telah dikunjungi.
2. Periksa apakah node saat ini memenuhi kriteria CSS selector yang diberikan.
3. Kunjungi setiap *child node* secara rekursif (atau dengan stack), dengan urutan dari child pertama hingga terakhir.
4. Lakukan backtracking ke parent node setelah semua child dari node saat ini telah dikunjungi.
5. Ulangi hingga seluruh node telah dikunjungi atau jumlah hasil yang diinginkan telah terpenuhi.

Kompleksitas waktu DFS adalah $O(V + E)$ di mana V adalah jumlah node (vertex) dan E adalah jumlah sisi (edge) pada graf. Pada pohon DOM, nilai $E = V - 1$, sehingga kompleksitas efektifnya adalah $O(V)$.

Kelebihan utama DFS dibandingkan BFS terletak pada efisiensi penggunaan memori. DFS hanya memerlukan ruang penyimpanan sebanding dengan kedalaman maksimum pohon ($O(d)$), karena ia hanya perlu menyimpan simpul-simpul pada jalur yang sedang ditelusuri dalam stack. Hal ini berbeda dengan BFS yang harus menyimpan seluruh simpul pada satu level tertentu dalam antrean, yang bisa sangat besar pada pohon yang lebar. Namun, perlu dicatat bahwa DFS tidak menjamin ditemukannya solusi pada lintasan terpendek jika terdapat lebih dari satu simpul yang memenuhi kriteria selector. Algoritma ini sangat efektif jika solusi atau elemen yang dicari terletak di bagian dalam (daerah yang jauh dari root) pada salah satu cabang pohon.

2.2.2. Breadth-First Search

Breadth-First Search (BFS) adalah algoritma traversal fundamental yang menelusuri graf secara berlapis atau per level. Algoritma ini bekerja dengan memulai pencarian dari sebuah node awal, lalu secara bertahap mengunjungi seluruh node tetangga yang berdekatan langsung dengannya. Setelah itu, algoritma akan melanjutkan penelusuran ke node tetangga dari tingkatan tersebut yang belum tereksplorasi, dan seterusnya berulang secara melebar hingga seluruh node di dalam graf berhasil dikunjungi.

Pada pohon DOM, BFS dimulai dari node root dan mengunjungi semua node pada level yang sama sebelum turun ke level berikutnya. Algoritma ini diimplementasikan menggunakan struktur data antrian (queue).

Langkah-langkah algoritma BFS pada pohon DOM adalah sebagai berikut:

1. Masukkan node root ke dalam antrian (queue) dan tandai sebagai telah dikunjungi.
2. Selama antrian tidak kosong, ambil node terdepan dari antrian.
3. Periksa apakah node tersebut memenuhi kriteria CSS selector yang diberikan.
4. Masukkan semua child node yang belum dikunjungi ke dalam antrian.
5. Ulangi hingga antrian kosong atau jumlah hasil yang diinginkan telah terpenuhi.

Sama halnya dengan DFS, kompleksitas waktu BFS berbanding lurus dengan jumlah simpul dan sisi, yakni $O(V + E)$ (Munir, 2026). Karena pohon DOM memiliki struktur di mana jumlah sisi $E = V - 1$, maka efektifitas waktu pemrosesannya dapat disederhanakan menjadi $O(V)$. Meskipun memiliki waktu eksekusi yang sepadan dengan DFS, BFS membutuhkan konsumsi memori yang jauh lebih besar. Hal ini terjadi karena BFS perlu menyimpan seluruh node pada level atau kedalaman tertentu secara bersamaan di dalam antrian (Cormen et al., 2009). Pada kasus terburuk (pohon yang sangat lebar), kebutuhan memori ini akan membesar secara eksponensial sebanding dengan batas percabangan maksimal pohon tersebut.

Namun, kelemahan pada memori tersebut diimbangi oleh kelebihan utama dan mutlak dari algoritma BFS: jaminan penemuan solusi paling optimal dan terdekat (Munir, 2026). BFS selalu memastikan bahwa elemen yang ditemukan pertama kali adalah elemen dengan kedalaman terkecil atau jalur terpendek dari root. Dalam konteks

pencarian DOM, sifat ini menjadikan BFS sangat superior dan cocok diandalkan ketika CSS selector yang ditargetkan kemungkinan besar menunjuk pada elemen-elemen kerangka utama (seperti header, navbar, atau kontainer utama suatu website) yang letaknya dangkal, sehingga program tidak perlu membuang waktu menelusuri ujung daun yang tidak relevan (Russell & Norvig, 2010).

2.2.3. *Lowest Common Ancestor* (LCA) melalui Binary Lifting

Dalam struktur data pohon, *Lowest Common Ancestor* (LCA) dari dua simpul u dan v didefinisikan sebagai simpul w yang terletak pada jalur dari kedua simpul tersebut menuju simpul akar (*root*) dan memiliki jarak terjauh dari akar. Secara konseptual, simpul ini merupakan leluhur bersama (*comon ancestor*) yang posisinya paling rendah di pohon. Apabila simpul u secara langsung maupun tidak langsung adalah leluhur dari simpul v , maka u itu sendiri bertindak sebagai LCA bagi kedua simpul tersebut.

Pencarian LCA menelusuri leluhur satu per satu dan seringkali tidak efisien, terutama pada struktur pohon yang dalam. Kompleksitas algoritma untuk LCA biasa $O(n)$. Untuk mengatasi hal ini, digunakan algoritma *Binary Lifting* yang mengoptimalkan pencarian melalui pendekatan eksponensial (perpangkatan dua). Metode ini didasarkan pada setiap bilangan dapat direpresentasikan oleh jumlah dari perpangkatan dua. Metode ini melakukan penelusuran graf melakukan "lompatan" besar ke atas pohon tanpa harus mengunjungi setiap simpul satu per satu. Algoritma ini beroperasi dalam dua fase utama: *preprocessing* dan *querying*.

Pada fase pra-pemrosesan, algoritma menelusuri pohon menggunakan metode *Depth-First Search* (DFS) untuk membangun tabel transisi (umumnya direpresentasikan sebagai *array* multidimensi). Tabel ini secara sistematis menyimpan informasi mengenai leluhur ke- 2^j dari setiap simpul. Pembangunan tabel dilakukan secara dinamis, di mana leluhur ke- 2^j dari sebuah simpul dievaluasi berdasarkan leluhur ke- 2^{j-1} dari simpul yang berada tepat 2^{j-1} tingkat di atasnya. Selain membangun tabel, fase DFS juga mencatat waktu masuk (t_{in}) dan waktu keluar (t_{out}) saat mengeksplorasi sebuah simpul. Metrik waktu ini esensial untuk memeriksa apakah sebuah simpul adalah leluhur dari simpul lainnya dalam kompleksitas waktu konstan $O(1)$. Proses *preprocessing* ini berjalan dengan kompleksitas waktu dan ruang $O(N \log N)$.

Pada fase eksekusi kueri, algoritma menerima permintaan untuk mencari LCA dari pasangan simpul (u,v) . Algoritma pertama-tama menggunakan penanda waktu t_{in} dan t_{out} untuk mengidentifikasi apakah salah satu simpul sudah menjadi leluhur bagi simpul lainnya. Jika tidak, algoritma akan memanfaatkan tabel transisi untuk memanjat dari simpul u menuju akar. Proses pemanjatan ini dilakukan dari lompatan eksponensial terbesar hingga terkecil, dengan syarat simpul tempat mendarat bukan merupakan leluhur dari v . Setelah siklus iterasi selesai, simpul u akan berhenti tepat satu tingkat di bawah LCA yang dicari. Simpul induk dari u pada titik inilah yang dikembalikan sebagai *Lowest Common Ancestor*. Berkat metode pemanjatan eksponensial ini, kompleksitas waktu untuk setiap kueri dapat direduksi secara signifikan menjadi $O(\log N)$.

2.3. Dasar Teori Lainnya

2.3.1. *Containerization* menggunakan Docker

Docker adalah platform open-source yang memungkinkan pengembang untuk mengemas, mendistribusikan, dan menjalankan aplikasi di dalam lingkungan yang terisolasi yang disebut *container*. Berbeda dengan virtualisasi tradisional yang mensimulasikan seluruh sistem operasi, container berbagi kernel sistem operasi host namun tetap memiliki isolasi pada level proses dan sistem berkas.

Menurut dokumentasi resmi Docker, sebuah *container* adalah unit standar dari perangkat lunak yang mengemas kode beserta seluruh dependensinya, sehingga aplikasi dapat berjalan secara konsisten di berbagai lingkungan komputasi. Docker menggunakan konsep Docker Image sebagai template read-only yang mendefinisikan isi dari container, dan Docker Container sebagai instansiasi yang dapat dijalankan dari image tersebut.

Komponen utama dalam ekosistem Docker antara lain:

- Dockerfile: berkas teks yang berisi instruksi untuk membangun Docker Image secara otomatis.
- Docker Image: template read-only yang berisi sistem operasi minimal, runtime, dependensi, dan kode aplikasi.
- Docker Container: instansi yang berjalan dari sebuah Docker Image.
- Docker Compose: alat untuk mendefinisikan dan menjalankan aplikasi *multicontainer* menggunakan file konfigurasi YAML.

Dalam konteks tugas besar ini, Docker digunakan untuk mengemas aplikasi frontend dan backend secara terpisah ke dalam container masing-masing, sehingga aplikasi dapat dijalankan di berbagai lingkungan tanpa perlu melakukan konfigurasi tambahan. Penggunaan Docker Compose memungkinkan orkestrasi kedua container secara bersamaan dengan satu perintah.

2.3.2. Virtual Machines melalui Microsoft Azure

Virtual Machine (VM) adalah emulasi dari sebuah sistem komputer yang berjalan di atas perangkat keras fisik dengan bantuan perangkat lunak yang disebut hypervisor. Setiap VM memiliki sistem operasi, penyimpanan, dan sumber daya komputasi sendiri yang terisolasi dari VM lain dan dari host fisiknya.

Berbeda dengan container yang berbagi kernel sistem operasi host, VM melakukan simulasi perangkat keras secara penuh sehingga memberikan tingkat isolasi yang lebih tinggi. Namun, VM membutuhkan sumber daya yang lebih besar karena setiap VM menjalankan sistem operasi lengkapnya sendiri.

Microsoft Azure Virtual Machine adalah layanan komputasi awan (cloud computing) yang menyediakan infrastruktur VM yang dapat dikonfigurasi sesuai kebutuhan. Dengan menggunakan Azure VM, aplikasi web dapat di-deploy sehingga dapat diakses secara publik melalui internet. Layanan ini menggunakan model pembayaran berbasis penggunaan (*pay-as-you-go*), sehingga biaya yang dikenakan sesuai dengan sumber daya yang digunakan.

2.3.3. *Multithreading* melalui Goroutines

Multithreading adalah teknik pemrograman di mana sebuah program menjalankan beberapa alur eksekusi (*thread*) secara bersamaan untuk meningkatkan efisiensi dan kecepatan pemrosesan, terutama pada sistem dengan prosesor multi-core.

Dalam bahasa pemrograman Go, konsep konkurensi diimplementasikan melalui mekanisme yang disebut goroutine. Goroutine adalah fungsi atau metode yang dijalankan secara konkuren dengan fungsi atau metode lainnya. Berbeda dengan thread sistem operasi konvensional, goroutine dikelola oleh *runtime* Go dan memiliki *overhead* yang jauh lebih kecil, sehingga sebuah program Go dapat menjalankan ribuan goroutine secara bersamaan dengan efisien.

Secara umum, goroutine memiliki karakteristik berikut

- Ringan (lightweight): Goroutine memulai dengan ukuran stack yang kecil (~2KB) dan dapat tumbuh secara dinamis sesuai kebutuhan
- Dikelola oleh Go runtime: Goroutine dijadwalkan oleh Go scheduler menggunakan model M:N (banyak Goroutine dipetakan ke sejumlah thread OS)
- Komunikasi melalui channel: Go mendorong komunikasi antar Goroutine menggunakan channel.
- Sinkronisasi dengan sync package: Go menyediakan primitif sinkronisasi seperti `sync.Mutex`, `sync.WaitGroup`, dan `sync.RWMutex` untuk mengatur akses konkuren ke sumber daya yang dibagi.

Dalam konteks traversal pohon DOM, multithreading dengan Goroutine dapat dimanfaatkan untuk memparalelkan proses BFS maupun DFS, di mana beberapa Goroutine dapat menelusuri cabang-cabang pohon yang berbeda secara bersamaan. Namun, perlu diperhatikan mekanisme sinkronisasi yang tepat untuk menghindari terjadinya *race condition* saat beberapa Goroutine mengakses atau memodifikasi data yang sama.

BAB 3

APLIKASI BFS DAN DFS

Dalam bagian ini, akan dibahas bagaimana algoritma Breadth-First Search (BFS) dan Depth-First Search (DFS) diimplementasikan dalam aplikasi untuk melakukan penelusuran (traversal) pada pohon DOM yang telah dibangun dari dokumen HTML serta menentukan node yang dipilih relatif terhadap node lain menggunakan selektor CSS.

3.1. Algoritma Breadth-First Search

Dalam aplikasi ini, algoritma Breadth-First Search (BFS) digunakan untuk menelusuri pohon DOM secara berlapis (level-order). BFS sangat efektif untuk menemukan elemen yang terletak pada kedalaman terkecil dari root terlebih dahulu. Implementasi BFS dalam tugas ini menggunakan struktur data *queue* untuk menyimpan node yang akan dikunjungi.

Setiap kali sebuah node dikeluarkan dari *queue*, aplikasi akan menjalankan fungsi matcher untuk memeriksa apakah node tersebut sesuai dengan selektor CSS yang dicari. Jika sesuai, node tersebut ditambahkan ke daftar hasil. Seluruh anak dari node tersebut kemudian dimasukkan ke dalam antrean untuk diproses pada iterasi berikutnya. Berikut adalah snippet kode yang diimplementasikan untuk penelusuran BFS.

3.1.1. Breadth-First Search secara Sekuensial

Pada implementasi sekuensial, penelusuran dilakukan menggunakan struktur data antrean (queue) secara iteratif. Node yang diambil dari depan antrean diperiksa terhadap selektor CSS, kemudian semua anak langsungnya dimasukkan ke belakang antrean. Proses ini menjamin urutan kunjungan yang melebar secara konsisten.

3.1.2. Breadth-First Search secara Konkuren

Breadth-First Search varian konkuren mengoptimalkan pemrosesan pada setiap level kedalaman pohon DOM dengan memproses seluruh node pada level yang sama secara paralel menggunakan Goroutines. Sinkronisasi antar node pada level yang sama dilakukan menggunakan `sync.WaitGroup`, memastikan seluruh pengecekan selesai sebelum algoritma mengumpulkan node untuk level berikutnya.

3.2. Algoritma Depth-First Search

Seperti yang sudah dijelaskan sebelumnya, Depth-First Search melakukan penelusuran sedalam mungkin pada satu cabang yang dipilih sebelum pada akhirnya melakukan *backtracking*. Hal ini bermanfaat dalam penelusuran DOM Tree apabila ingin mencari elemen-elemen pertama yang sesuai dengan selektor CSS dengan lebih mementingkan kedalaman terdalam pada cabang awal dibandingkan elemen yang terdapat pada tingkatan rendah pada BFS.

Algoritma DFS yang dipakai adalah algoritma yang menggunakan struktur data *stack* yang bersifat LIFO (Last-In-First-Out). Dengan menggunakan *stack*, urutan(berdasarkan kedalaman) dari elemen yang ditelusuri dipelihara dan akan mementingkan elemen yang sesuai dengan selektor CSS pada jalur-jalur awal DOM Tree.

3.2.1. Depth-First Search secara Sekuensial

Untuk mempertahankan urutan pembacaan dari kiri ke kanan sesuai standar dokumen HTML, anak-anak dari sebuah node dimasukkan ke dalam stack dengan urutan terbalik. Dengan demikian, anak pertama akan berada di posisi teratas dan diproses lebih dulu.

3.2.2. Depth-First Search secara Konkuren

Perbedaan mendasar pada DFS konkuren adalah pembagian tugas berdasarkan sub-pohon (*subtree parallelism*). Alih-alih melakukan paralelisme di setiap level seperti pada BFS, algoritma ini memicu Goroutine untuk setiap anak langsung dari root. Setiap Goroutine tersebut kemudian menjalankan DFS sekuensial pada sub-pohon masing-masing secara independen. Hal ini sangat efektif jika pohon DOM memiliki beberapa cabang utama yang sangat dalam.

BAB 4

IMPLEMENTASI DAN PENGUJIAN

4.1. Implementasi

Bagian ini akan merincikan alat dan teknologi yang digunakan dalam pengembangan aplikasi, serta pengorganisasian modul-modul di dalam proyek ini.

4.1.1. Tech Stack

Untuk menghasilkan aplikasi web yang dapat melakukan *web scraping* pada HTML suatu situs, menghasilkan DOM Tree yang sesuai, dan melakukan penelusuran baik secara sekuensial maupun konkuren serta menampilkan hasil *scraping* dan penelusuran dengan interaktif dan *readable*, dilakukan pemilihan *tech stack* untuk *frontend* dan *backend* aplikasi.

4.1.1.1. Backend

Backend dibangun menggunakan bahasa pemrograman Go, yang dipilih karena efisiensinya dalam menangani konkurensi (melalui Goroutines) serta kemampuannya menghasilkan binary tunggal yang ringan.

- Go (Golang): Digunakan sebagai bahasa utama karena performa eksekusinya yang mendekati C++, namun dengan sintaksis yang lebih modern.
- Gin Gonic: Web framework HTTP yang digunakan untuk membangun API. Gin dipilih karena kecepatannya yang luar biasa (memiliki performa routing yang sangat baik) dan dokumentasi yang lengkap.
- Go HTML Parser (net/html): Pustaka standar dari Go untuk memproses dokumen HTML menjadi struktur pohon.
- Air: Digunakan sebagai alat *live reload* selama pengembangan untuk mempercepat dan mempermudah pengembangan.

4.1.1.2. Frontend

Pemilihan *techstack* untuk *frontend* dilakukan dengan mempertimbangkan pengalaman pengguna yang reaktif dan visualisasi data yang interaktif.

- React 19: Pustaka utama untuk membangun antarmuka pengguna berbasis komponen.
- TanStack Start: Framework React generasi terbaru yang mendukung Server-Side Rendering (SSR) dan optimasi performa full-stack.
- Tailwind CSS v4: Digunakan untuk styling antarmuka secara langsung melalui class *utility* yang mempercepat pengembangan.
- React D3 Tree: Pustaka khusus untuk merender visualisasi pohon DOM secara interaktif, memungkinkan pengguna untuk melakukan zoom, pan, dan melihat hubungan antar node dengan jelas.
- Bun: Digunakan sebagai runtime dan package manager karena kecepatannya yang melampaui [Node.js/NPM](https://nodejs.org/en/).

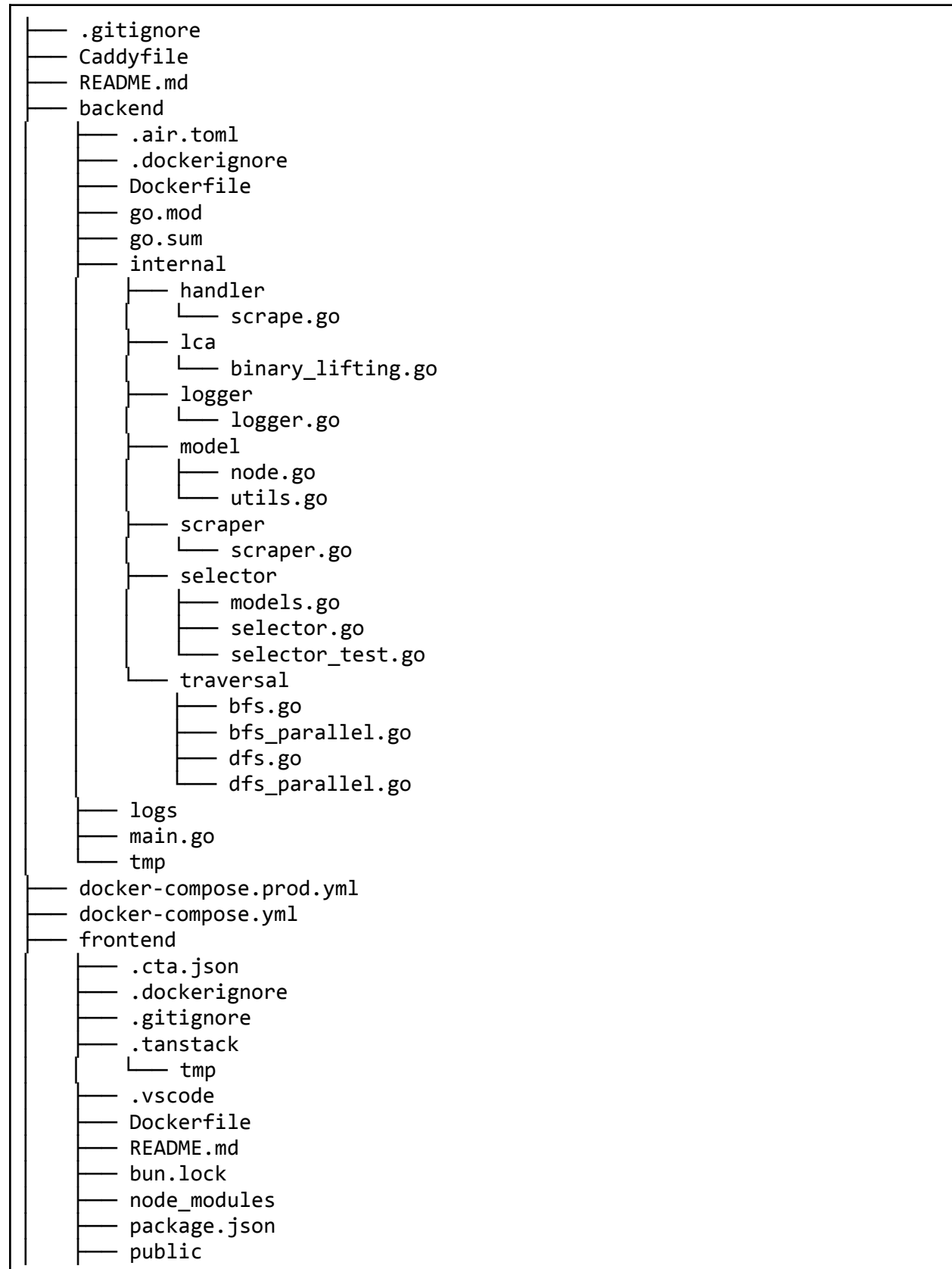
4.1.1.3. Infrastruktur dan Deployment

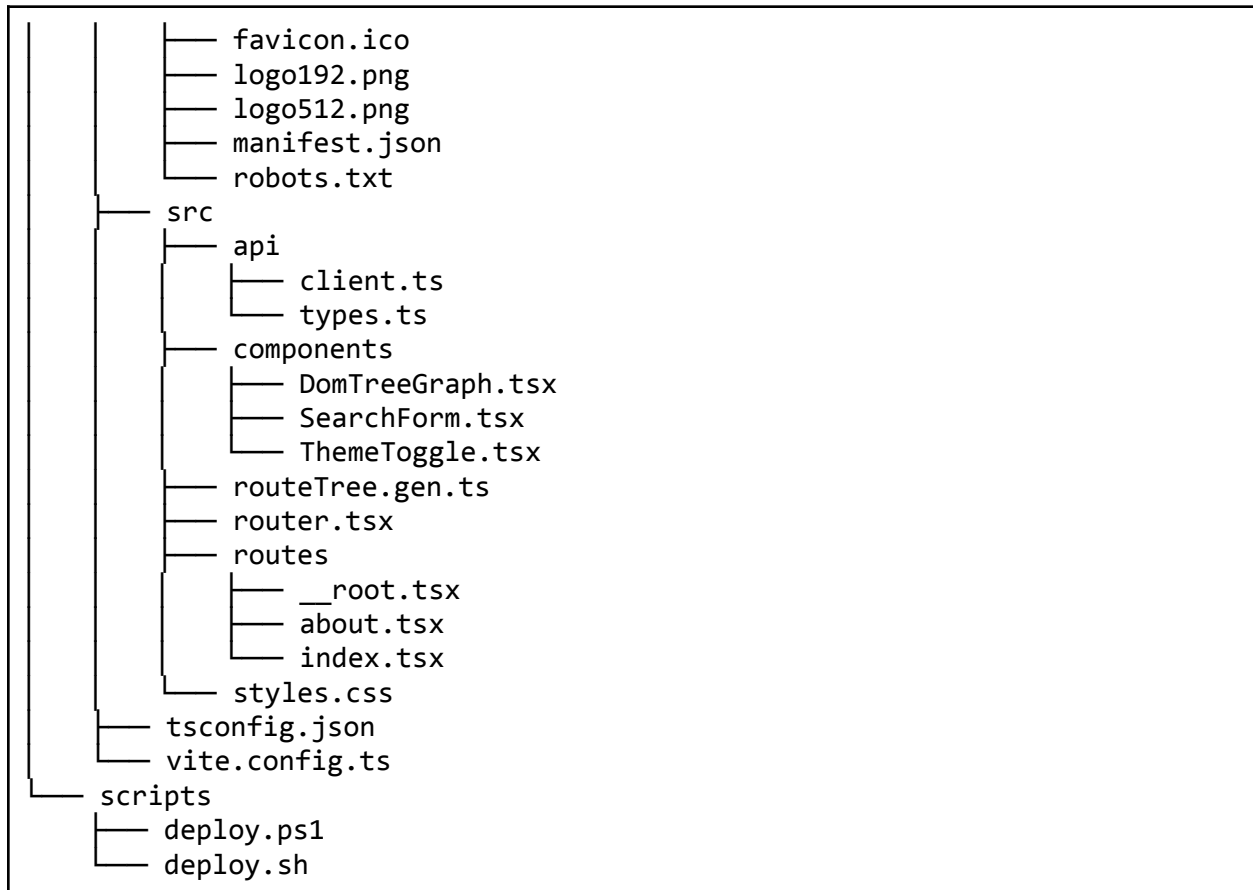
- Docker & Docker Compose: Digunakan untuk melakukan kontainerisasi aplikasi (*frontend*, *backend*, dan Caddy) agar dapat dijalankan secara konsisten di *environment* mana pun.
- Caddy: Berperan sebagai *reverse proxy* dan *web server* pada lingkungan produksi karena kemudahannya dalam konfigurasi dan manajemen sertifikat SSL otomatis.

4.1.2. Struktur Proyek

Proyek ini dibagi menjadi dua bagian utama (Frontend dan Backend) untuk memisahkan tanggung jawab antara logika pemrosesan data dan presentasi. Berikut adalah gambaran pohon struktur proyek secara garis besar.

```
Tubes2_TimsesDewaPetir
├── .env.example
├── .git
```





4.1.3. Kode Program

4.1.3.1. Implementasi Pengambilan dan Parsing HTML (scraper.go)

Tahap pertama dalam berjalannya program adalah mengambil dokumen HTML mentah dari website target dan mengubahnya (parsing) menjadi struktur data pohon. Logika ini ditangani oleh paket scraper yang terdiri dari dua fungsi utama: pengambilan jaringan dan konstruksi graf.

- Pengambilan Dokumen (Fetching)

Fungsi FetchHTML bertugas melakukan permintaan HTTP GET ke URL target. Untuk menghindari pemblokiran oleh sistem keamanan website (seperti bot protection), permintaan tidak dikirim secara kosong. Program menyisipkan header jaringan yang mensimulasikan permintaan dari peramban nyata.

```
// Set a realistic User-Agent to bypass bot protection
req.Header.Set("User-Agent", "Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36")
```

```

req.Header.Set("Accept",
"text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8")
req.Header.Set("Accept-Language", "en-US,en;q=0.5")

```

- Konstruksi Pohon DOM (Parsing)

Setelah teks HTML mentah (string) didapatkan, fungsi Parse akan membedah teks tersebut menjadi token secara berurutan menggunakan `html.NewTokenizer`. Karena struktur HTML secara alami merupakan pohon dengan kedalaman bertingkat, proses konstruksi ini diimplementasikan menggunakan algoritma berbasis Struktur Data Stack (Tumpukan).

```

case html.StartTagToken, html.SelfClosingTagToken:
    t := z.Token()
    counter++
    node := &model.DOMNode{
        ID:      counter,
        Tag:      t.Data,
        Depth:    len(stack),
        Attributes: make(map[string]string),
    }
    for _, a := range t.Attr {
        node.Attributes[a.Key] = a.Val
    }

    parent := stack[len(stack)-1]
    node.Parent = parent

    if prev, ok := lastAppended[parent.ID]; ok {
        node.PreviousSibling = prev
    }
    parent.Children = append(parent.Children, node)
    lastAppended[parent.ID] = node

    if tt == html.StartTagToken {
        stack = append(stack, node)
        lastAppended[node.ID] = nil
    }

```

Program selalu menganggap elemen teratas (ujung) dari stack sebagai induk (parent) dari node yang sedang diproses. Jika program menemukan tag pembuka (seperti `<div>`), elemen tersebut didorong (push) ke dalam stack. Sebaliknya, jika menemukan tag

penutup (`</div>`), program akan melakukan pop (mengeluarkan elemen dari stack).

Salah satu implementasi strategis di sini adalah penggunaan map `lastAppended`. Map ini mencatat anak terakhir yang dimasukkan ke suatu parent. Hal ini memungkinkan program untuk mengisi pointer `PreviousSibling` dengan kompleksitas waktu $O(1)$. Data relasi sibling ini sangat vital dan nantinya akan digunakan oleh algoritma DFS/BFS untuk mengevaluasi CSS Selector kombinasional seperti `+` (elemen bersebelahan) dan `~` (saudara umum).

Di akhir fungsi, program melakukan pembersihan dengan menyeleksi root utama (mencari tag `<html>`) dan memperbaiki kalkulasi tingkat kedalaman (depth) menggunakan fungsi rekursif `fixDepths`.

4.1.3.2. Representasi Pohon DOM (node.go)

Dalam mengimplementasikan penelusuran pohon DOM, hal pertama yang dilakukan adalah mendefinisikan representasi perangkat lunak dari sebuah node HTML. Pada berkas `node.go`, model data dibangun menggunakan sebuah struktur (struct) bernama `Node`. Struktur ini dirancang untuk menyimpan atribut-atribut penting dari elemen HTML yang nantinya akan digunakan oleh algoritma traversal dan fungsionalitas highlighting.

```
// individual DOM Node
type DOMNode struct {
    ID            int           `json:"id"`
    Tag           string        `json:"tag"`
    Attributes    map[string]string `json:"attributes"`
    Text         string        `json:"text"`
    Depth        int           `json:"depth"`
    Children     []*DOMNode    `json:"children"`
    Parent       *DOMNode     `json:"- "` // Added for Combinators
    PreviousSibling *DOMNode    `json:"- "` // Added for Combinators
}
```

- ID & ParentID: Setiap node diberikan pengidentifikasi unik berupa integer. ParentID sangat krusial untuk membantu algoritma melakukan backtracking atau rekonstruksi jalur dari node anak kembali ke akar (root).
- Tag: Menyimpan nama tag HTML (seperti div, p, h1). Ini adalah parameter utama yang diperiksa saat melakukan pencarian menggunakan Tag Selector.
- Attrs: Menggunakan tipe data map[string]string untuk menyimpan pasangan kunci-nilai dari atribut HTML (misalnya class="container" atau id="header"). Struktur map dipilih agar proses pencarian atribut spesifik oleh modul selector menjadi lebih efisien ($O(1)$ untuk akses kunci).
- Content: Menyimpan teks yang berada di dalam tag tersebut.
- Children: Sebuah slice (larik dinamis) yang berisi pointer ke objek Node lainnya. Inilah yang membentuk struktur hirarki pohon, di mana satu node dapat memiliki banyak anak (simpul cabang).
- IsMatched: Sebuah flag boolean yang berfungsi sebagai penanda. Jika sebuah node sesuai dengan CSS Selector yang dicari oleh pengguna, nilai ini akan diubah menjadi true. Atribut ini nantinya dikirim ke frontend dalam format JSON untuk memberikan efek visual (highlight) pada pohon yang ditampilkan.

Model ini memastikan bahwa seluruh informasi penting dari sebuah elemen web tetap terjaga meskipun strukturnya sudah berubah dari teks linear menjadi objek pohon yang kompleks.

4.1.3.3. Algoritma Evaluasi Selektor (selector.go)

Setelah struktur pohon direpresentasikan melalui model DOMNode, program memerlukan sebuah mekanisme (Goal Test) untuk mengevaluasi apakah simpul yang sedang diperiksa oleh algoritma BFS/DFS cocok dengan CSS Selector yang ditargetkan pengguna. Mengingat spesifikasi selector pada CSS sangat kompleks dan mendukung sintaks turunan, logika ini dikapsulasi dalam package selector.

Pendekatan yang digunakan adalah Polimorfisme melalui interface Matcher, di mana string selektor mentah akan diurai (parsing) ke dalam objek-objek matcher terstruktur menggunakan fungsi ParseSelector.

```
func (g *GroupMatcher) Match(node *model.DOMNode) bool {
    for _, m := range g.Matchers {
        if m.Match(node) {
            return true
        }
    }
    return false
}

func (c *CombinatorMatcher) Match(node *model.DOMNode) bool {
    if !c.Right.Match(node) {
        return false
    }
    switch c.Op {
    case '>':
        return node.Parent != nil && c.Left.Match(node.Parent)
    case ' ':
        curr := node.Parent
        for curr != nil {
            if c.Left.Match(curr) {
                return true
            }
            curr = curr.Parent
        }
    case '+':
        return node.PreviousSibling != nil &&
c.Left.Match(node.PreviousSibling)
    case '~':
        curr := node.PreviousSibling
        for curr != nil {
            if c.Left.Match(curr) {
                return true
            }
            curr = curr.PreviousSibling
        }
    }
    return false
}

func (m *CompoundMatcher) Match(node *model.DOMNode) bool {
    if m.Tag != "" && m.Tag != "*" && m.Tag != node.Tag {
        return false
    }
    if m.ID != "" && node.Attributes["id"] != m.ID {
```

```
        return false
    }
    for _, cls := range m.Classes {
        if !hasClass(node.Attributes["class"], cls) {
            return false
        }
    }
    for _, attr := range m.Attributes {
        val, exists := node.Attributes[attr.Key]
        if !exists {
            return false
        }
        if attr.Operator == "" {
            continue
        }
        switch attr.Operator {
        case "=":
            if val != attr.Value {
                return false
            }
        case "~=":
            if !hasWord(val, attr.Value) {
                return false
            }
        case "^=":
            if !strings.HasPrefix(val, attr.Value) {
                return false
            }
        case "$=":
            if !strings.HasSuffix(val, attr.Value) {
                return false
            }
        case "*=":
            if !strings.Contains(val, attr.Value) {
                return false
            }
        }
    }
    return true
}

// ParseSelector compiles a CSS selector string into Matcher
func ParseSelector(selector string) Matcher {
    selector = strings.TrimSpace(selector)
    if selector == "" {
        return &CompoundMatcher{Tag: ""}
    }
}
```

```
parts := splitOutsideBrackets(selector, ',')
if len(parts) > 1 {
    group := &GroupMatcher{}
    for _, part := range parts {
        group.Matchers = append(group.Matchers,
ParseSelector(part))
    }
    return group
}

leftStr, rightStr, combinator, found := splitLastCombinator(selector)
if found {
    return &CombinatorMatcher{
        Left: ParseSelector(leftStr),
        Right: parseCompound(rightStr),
        Op:    combinator,
    }
}

return parseCompound(selector)
}

func parseCompound(s string) Matcher {
    s = strings.TrimSpace(s)
    if s == "" {
        return &CompoundMatcher{Tag: ""}
    }
    m := &CompoundMatcher{}

    i := 0
    for i < len(s) {
        c := s[i]
        if c == '.' {
            end := findTokenEnd(s, i+1)
            m.Classes = append(m.Classes, s[i+1:end])
            i = end
        } else if c == '#' {
            end := findTokenEnd(s, i+1)
            m.ID = s[i+1 : end]
            i = end
        } else if c == '[' {
            end := strings.Index(s[i:], "]")
            if end == -1 {
                end = len(s[i:])
            }
            attrStr := s[i+1 : i+end]
            m.Attributes = append(m.Attributes, parseAttr(attrStr))
            i = i + end + 1
        }
    }
}
```

```

        } else {
            end := findTokenEnd(s, i)
            m.Tag = s[i:end]
            i = end
        }
    }
    return m
}

func parseAttr(s string) AttributeMatch {
    match := AttributeMatch{}
    ops := []string{"~=", "^=", "$=", "*=", "="}
    for _, op := range ops {
        if idx := strings.Index(s, op); idx != -1 {
            match.Key = strings.TrimSpace(s[:idx])
            match.Operator = op
            match.Value =
strings.Trim(strings.TrimSpace(s[idx+len(op):]), `"'`)
            return match
        }
    }
    match.Key = strings.TrimSpace(s)
    return match
}

```

Program membagi skenario evaluasi ke dalam tiga implementasi Match yang berbeda, yang masing-masing menangani tingkat hierarki CSS yang berbeda:

- GroupMatcher (Selektor Majemuk): Menangani kasus pemisahan koma (.). Fungsi Match-nya bekerja dengan prinsip logika OR. Jika salah satu sub-selektor (misal: node memenuhi selektor A atau B) mengembalikan nilai true, maka node tersebut dianggap sebagai kecocokan.
- CombinatorMatcher (Selektor Relasional): Evaluasi ini menangani hierarki antar node (Child, Descendant, Sibling). Program mengevaluasi elemen dari "kanan ke kiri".
 - Misalnya, untuk selektor `div > p`, program pertama-tama memastikan apakah node saat ini adalah elemen target akhir (p).

- Jika ya, program memeriksa status relasi leluhurnya melalui `node.Parent` (untuk `>`) atau melacak mundur melalui `node.PreviousSibling` (untuk `+` atau `~`) untuk memastikan aturan kombinasinya terpenuhi.
- **CompoundMatcher (Selektor Tunggal):** Ini adalah evaluasi paling dasar. Program membedah atribut node dan mencocokkannya dengan ID (`#`), Class (`.`), awalan/akhiran string, serta nama tag HTML. Pada level ini, pencocokan class juga telah mengantisipasi kondisi elemen dengan multi-kelas dengan menggunakan pemisahan spasi otomatis `strings.Fields`.

Dengan arsitektur parser ini, beban evaluasi string dilakukan hanya satu kali di awal, sehingga proses traversal graf yang dilakukan oleh algoritma utama (BFS/DFS) dapat beroperasi dengan jauh lebih efisien.

4.1.3.4. Implementasi BFS dan DFS

Mengacu pada rancangan konseptual algoritma penelusuran BFS dan DFS yang telah dijabarkan pada Bab 3, berikut adalah cuplikan kode masing-masing.

BFS secara sekuensial

```
func BFS(root *model.DOMNode, match func(*model.DOMNode) bool, limit int)
([]*model.DOMNode, []model.TraversalStep, []model.AnimationFrame, int) {
    var matches []*model.DOMNode
    var log []model.TraversalStep
    var frames []model.AnimationFrame
    visited := 0

    if root == nil {
        return matches, log, frames, visited
    }

    queue := []*model.DOMNode{root}
    var matchedIDs []int

    for len(queue) > 0 {
        node := queue[0]
        queue = queue[1:]

        visited++
    }
}
```

```

        matched := match(node)
        parentID := -1
        if node.Parent != nil {
            parentID = node.Parent.ID
        }
        log = append(log, model.TraversalStep{
            NodeID:    node.ID,
            ParentID:  parentID,
            Tag:       node.Tag,
            Depth:     node.Depth,
            Matched:   matched,
        })

        if matched {
            if limit == 0 || len(matches) < limit {
                matches = append(matches, node)
                matchedIDs = append(matchedIDs, node.ID)
            }
        }

        queue = append(queue, node.Children...)

        queueIDs := make([]int, len(queue))
        for i, n := range queue {
            queueIDs[i] = n.ID
        }

        copiedMatchedIDs := make([]int, len(matchedIDs))
        copy(copiedMatchedIDs, matchedIDs)

        frames = append(frames, model.AnimationFrame{
            Step:       len(frames),
            ActiveID:   node.ID,
            QueueIDs:   queueIDs,
            StackIDs:   []int{},
            MatchedIDs: copiedMatchedIDs,
        })
    }

    return matches, log, frames, visited
}

```

BFS secara konkuren

```

func BFSParallel(root *model.DOMNode, match func(*model.DOMNode) bool,
    limit int) (
    []*model.DOMNode, []model.TraversalStep, []model.AnimationFrame, int,

```



```

) {
    if root == nil {
        return nil, nil, nil, 0
    }

    var (
        allMatches    []*model.DOMNode
        allLog         []model.TraversalStep
        allFrames      []model.AnimationFrame
        matchedIDs     []int
        visitedCount   int
    )

    currentLevel := []*model.DOMNode{root}

    for len(currentLevel) > 0 {
        levelSize := len(currentLevel)
        results := make([]bfsNodeResult, levelSize)

        var wg sync.WaitGroup
        wg.Add(levelSize)
        for i, node := range currentLevel {
            i, node := i, node // capture loop variables
            go func() {
                defer wg.Done()
                matched := match(node) // pure CSS matcher –
goroutine-safe
                parentID := -1
                if node.Parent != nil {
                    parentID = node.Parent.ID
                }
                results[i] = bfsNodeResult{
                    node:    node,
                    matched: matched,
                    step: model.TraversalStep{
                        NodeID:    node.ID,
                        ParentID: parentID,
                        Tag:        node.Tag,
                        Depth:    node.Depth,
                        Matched:  matched,
                    },
                },
            }()
        }
        wg.Wait()

        var nextLevel []*model.DOMNode
        for _, res := range results {

```

```

        allLog = append(allLog, res.step)
        if res.matched && (limit == 0 || len(allMatches) < limit)
{
            allMatches = append(allMatches, res.node)
            matchedIDs = append(matchedIDs, res.node.ID)
        }
        nextLevel = append(nextLevel, res.node.Children...)
    }

    childrenSoFar := 0
    for i, res := range results {
        childrenThisNode := len(res.node.Children)

        var queueSnap []int
        for _, n := range currentLevel[i+1:] {
            queueSnap = append(queueSnap, n.ID)
        }
        for _, n := range
nextLevel[:childrenSoFar+childrenThisNode] {
            queueSnap = append(queueSnap, n.ID)
        }
        childrenSoFar += childrenThisNode

        copiedMatchedIDs := make([]int, len(matchedIDs))
        copy(copiedMatchedIDs, matchedIDs)

        allFrames = append(allFrames, model.AnimationFrame{
            Step:      len(allFrames),
            ActiveID:   res.node.ID,
            QueueIDs:   queueSnap,
            StackIDs:   []int{},
            MatchedIDs: copiedMatchedIDs,
        })
    }

    visitedCount += levelSize
    currentLevel = nextLevel
}

return allMatches, allLog, allFrames, visitedCount
}

```

DFS secara sekuensial

```

func DFS(root *model.DOMNode, match func(*model.DOMNode) bool, limit int)
([]*model.DOMNode, []model.TraversalStep, []model.AnimationFrame, int) {
    var matches []*model.DOMNode

```

```

var log []model.TraversalStep
var frames []model.AnimationFrame
visited := 0

if root == nil {
    return matches, log, frames, visited
}

stack := []*model.DOMNode{root}
var matchedIDs []int

for len(stack) > 0 {
    // Pop from top of stack
    node := stack[len(stack)-1]
    stack = stack[:len(stack)-1]

    visited++

    matched := match(node)
    parentID := -1
    if node.Parent != nil {
        parentID = node.Parent.ID
    }
    log = append(log, model.TraversalStep{
        NodeID:    node.ID,
        ParentID:  parentID,
        Tag:       node.Tag,
        Depth:    node.Depth,
        Matched:  matched,
    })

    if matched {
        if limit == 0 || len(matches) < limit {
            matches = append(matches, node)
            matchedIDs = append(matchedIDs, node.ID)
        }
    }

    // Push children in reverse order so the first child is popped
    for i := len(node.Children) - 1; i >= 0; i-- {
        stack = append(stack, node.Children[i])
    }

    // Capture Animation Frame
    stackIDs := make([]int, len(stack))
    for i, n := range stack {
        stackIDs[i] = n.ID
    }
}

```

first

```

    }

    copiedMatchedIDs := make([]int, len(matchedIDs))
    copy(copiedMatchedIDs, matchedIDs)

    frames = append(frames, model.AnimationFrame{
        Step:      len(frames),
        ActiveID:   node.ID,
        QueueIDs:  []int{},
        StackIDs:   stackIDs,
        MatchedIDs: copiedMatchedIDs,
    })
}

return matches, log, frames, visited
}

```

DFS secara konkuren

```

func DFSParallel(root *model.DOMNode, match func(*model.DOMNode) bool,
limit int) (
    []*model.DOMNode, []model.TraversalStep, []model.AnimationFrame, int,
) {
    if root == nil {
        return nil, nil, nil, 0
    }
    if len(root.Children) <= 1 {
        return DFS(root, match, limit)
    }

    var (
        allMatches    []*model.DOMNode
        allLog         []model.TraversalStep
        allFrames      []model.AnimationFrame
        visitedCount  int
    )

    visitedCount = 1
    rootMatched := match(root)
    var rootMatchedIDs []int
    if rootMatched {
        allMatches = append(allMatches, root)
        rootMatchedIDs = append(rootMatchedIDs, root.ID)
    }
    allLog = append(allLog, model.TraversalStep{
        NodeID:  root.ID,
        ParentID: -1,
    })
}

```

```

        Tag:      root.Tag,
        Depth:    root.Depth,
        Matched:  rootMatched,
    })

    rootStackIDs := make([]int, len(root.Children))
    for i, c := range root.Children {
        rootStackIDs[i] = c.ID
    }
    copiedRoot := make([]int, len(rootMatchedIDs))
    copy(copiedRoot, rootMatchedIDs)
    allFrames = append(allFrames, model.AnimationFrame{
        Step:      0,
        ActiveID:   root.ID,
        QueueIDs:   []int{},
        StackIDs:    rootStackIDs,
        MatchedIDs: copiedRoot,
    })

    children := root.Children
    subtreeResults := make([]dfsSubtreeResult, len(children))
    var wg sync.WaitGroup
    wg.Add(len(children))
    for i, child := range children {
        i, child := i, child
        go func() {
            defer wg.Done()
            subtreeResults[i] = sequentialDFSSubtree(child, match)
        }()
    }
    wg.Wait()

    prefixIDs := make([][]int, len(children))
    running := make([]int, len(rootMatchedIDs))
    copy(running, rootMatchedIDs)
    for i, sub := range subtreeResults {
        prefixIDs[i] = make([]int, len(running))
        copy(prefixIDs[i], running)
        for _, m := range sub.matches {
            running = append(running, m.ID)
        }
    }

    frameOffsets := make([]int, len(children))

    for i, sub := range subtreeResults {
        visitedCount += sub.visited
        allLog = append(allLog, sub.log...)
    }

```

```

        allMatches = append(allMatches, sub.matches...)

        frameOffsets[i] = len(allFrames)
        for _, f := range sub.frames {
            f.Step = len(allFrames)
            allFrames = append(allFrames, f)
        }
    }

    for i, sub := range subtreeResults {
        prefix := prefixIDs[i]
        for j := range sub.frames {
            gIdx := frameOffsets[i] + j
            local := allFrames[gIdx].MatchedIDs
            combined := make([]int, 0, len(prefix)+len(local))
            combined = append(combined, prefix...)
            combined = append(combined, local...)
            allFrames[gIdx].MatchedIDs = combined
        }
    }

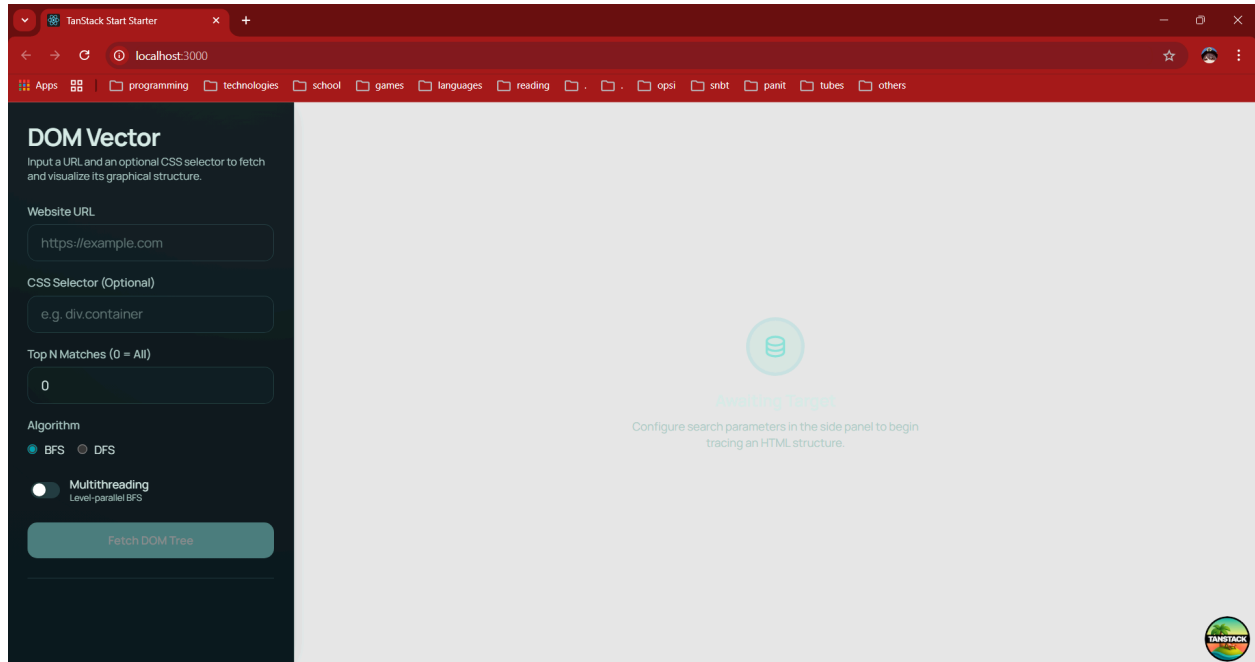
    if limit > 0 && len(allMatches) > limit {
        allMatches = allMatches[:limit]
    }

    return allMatches, allLog, allFrames, visitedCount
}

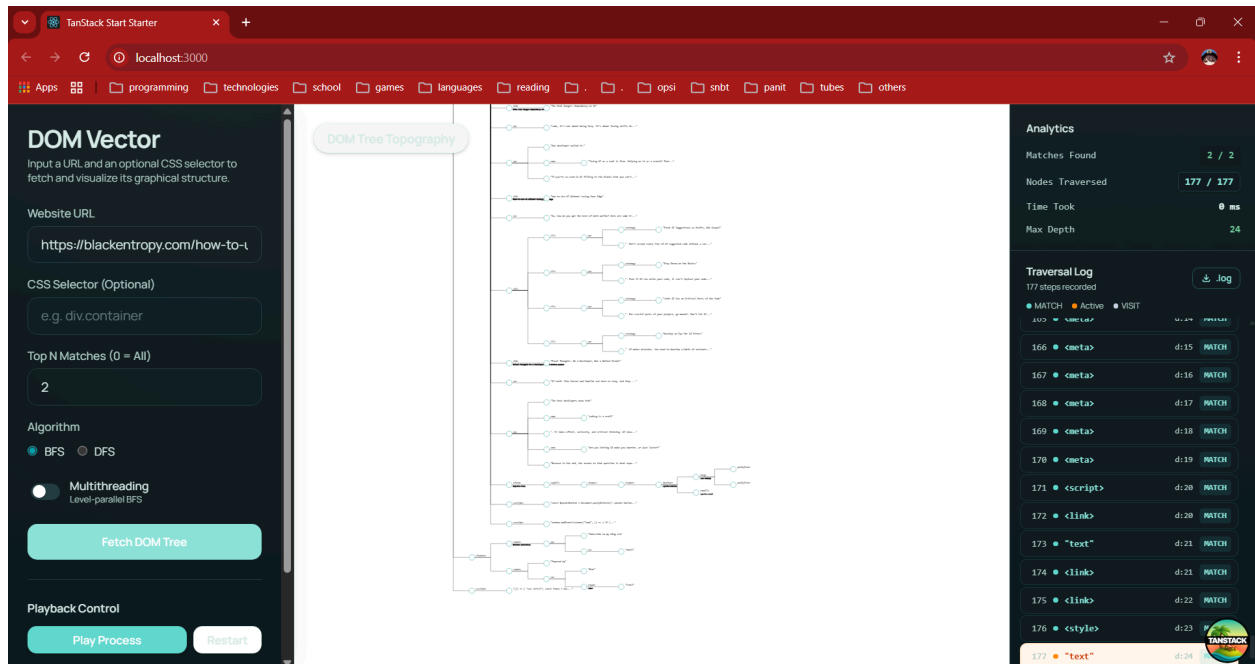
```

4.1.3.5. Antarmuka Pengguna (Frontend)

Antarmuka pengguna (frontend) pada aplikasi ini dikembangkan menggunakan lingkungan eksekusi Bun dan bundler Vite untuk menghasilkan Single Page Application (SPA) yang responsif. Dalam arsitektur sistem, bagian ini bertindak sebagai klien murni yang bertugas mengumpulkan parameter pencarian dari pengguna, mengirimkan kueri ke endpoint backend (API), dan memvisualisasikan respons JSON yang dikembalikan.



Pada halaman utama, pengguna disediakan formulir masukan yang mencakup URL target atau teks HTML mentah, CSS Selector yang ingin dicari, pilihan algoritma penelusuran (Breadth-First Search atau Iterative Deepening Depth-First Search), serta konfigurasi jumlah hasil yang ingin ditampilkan.



Setelah proses komputasi di backend selesai, antarmuka akan merender struktur data pohon DOM secara interaktif. Untuk memenuhi spesifikasi visualisasi, frontend melakukan pemrosesan data untuk menampilkan komponen berikut:

- Visualisasi Pohon dan Kedalaman: Menggambarkan hierarki parent-child dari elemen HTML beserta keterangan batas kedalaman maksimumnya.
- Animasi dan Highlight Jalur: Node yang dieksplorasi oleh algoritma dan node target yang cocok dengan CSS Selector ditandai dengan warna kontras (sorotan visual) agar jalur penelusuran (traversal path) dapat diamati dengan jelas.
- Metrik Performa dan Traversal Log: Aplikasi menampilkan secara akurat waktu eksekusi komputasi algoritma (dalam milidetik) dan total simpul (node) yang dikunjungi. Selain itu, terdapat panel traversal log yang merekam tahapan atau urutan kunjungan simpul secara kronologis.

4.1.3.6. Implementasi Deployment dan Orkestrasi Kontainer

Untuk memastikan aplikasi dapat diakses secara publik dan memiliki lingkungan eksekusi yang konsisten, sistem ini tidak hanya dijalankan secara lokal, melainkan juga di-deploy ke ranah cloud menggunakan infrastruktur Microsoft Azure Virtual Machine (VM). Seluruh proses pengemasan dan orkestrasi layanan dilakukan menggunakan teknologi Docker.

Implementasi deployment ini terbagi ke dalam tiga komponen utama:

- Kontainerisasi Layanan (Dockerization): Baik layanan backend (Go) maupun frontend (Bun/Vite) dikemas ke dalam image Docker masing-masing melalui instruksi pada Dockerfile. Pendekatan multi-stage build diterapkan pada backend untuk menjaga ukuran image produksi tetap sekecil mungkin.

- Orkestrasi dengan Docker Compose: Pengelolaan kontainer dilakukan melalui konfigurasi `docker-compose.prod.yml`. File ini bertugas untuk menyatukan frontend, backend, dan layanan proxy dalam satu jaringan virtual (bridge network) yang sama di dalam Azure VM, sehingga mereka dapat saling berkomunikasi secara aman.
- Reverse Proxy menggunakan Caddy: Alih-alih mengekspos port frontend dan backend secara langsung ke internet, program menggunakan Caddy sebagai web server sekaligus reverse proxy. Melalui aturan yang didefinisikan pada Caddyfile, lalu lintas (traffic) HTTP yang masuk dari luar VM akan diarahkan ke frontend untuk memuat UI, sementara permintaan dengan endpoint spesifik (seperti `/api/*`) akan diteruskan langsung ke kontainer backend.

Pendekatan arsitektur ini memastikan aplikasi berjalan secara terisolasi, aman, dan mempermudah proses pembaruan (re- build) tanpa harus mengganggu konfigurasi sistem operasi asli dari VM yang digunakan.

4.2. Pengujian

4.2.1. Test Case 1

Input: `http://example.com`

Output:

- BFS

DOM Vector

Input a URL and an optional CSS selector to fetch and visualize its graphical structure.

Website URL

CSS Selector (Optional)

Top N Matches (0 = All)

Algorithm

☒ BFS
 ☐ DFS

☐ Multithreading
 Level-parallel BFS

Playback Control

DOM Tree Topography

16 nodes

Analytics

Matches Found: 16 / 16

Nodes Traversed: 16 / 16

Time Took: 0 ms

Max Depth: 5

Traversal Log

16 steps recorded

☒ MATCH
 ☐ Active
 ☐ VISIT

Step	Node	Depth	Status
5	<meta>	d:2	MATCH
6	<div>	d:2	MATCH
7	"text"	d:3	MATCH
8	<style>	d:3	MATCH
9	<h1>	d:3	MATCH
10	<p>	d:3	MATCH
11	<p>	d:3	MATCH
12	"text"	d:4	MATCH
13	"text"	d:4	MATCH
14	"text"	d:4	MATCH
15	<a>	d:4	MATCH
16	"text"	d:5	MATCH

Analytics

Matches Found: 16 / 16

Nodes Traversed: 16 / 16

Time Took: 0 ms

Max Depth: 5

Traversal Log

16 steps recorded

☒ MATCH
 ☐ Active
 ☐ VISIT

Step	Node	Depth	Status
5	<meta>	d:2	MATCH
6	<div>	d:2	MATCH
7	"text"	d:3	MATCH
8	<style>	d:3	MATCH
9	<h1>	d:3	MATCH
10	<p>	d:3	MATCH
11	<p>	d:3	MATCH
12	"text"	d:4	MATCH
13	"text"	d:4	MATCH
14	"text"	d:4	MATCH
15	<a>	d:4	MATCH
16	"text"	d:5	MATCH

- BFS dengan multithreading

[illegible]

Analytics

Matches Found

16 / 16

Nodes Traversed

16 / 16

Time Took

0 ms

Max Depth

5

Traversal Log

16 steps recorded

MATCH

Active

VISIT

	STEP	TYPE
5	• <meta>	d:2 MATCH
6	• <div>	d:2 MATCH
7	• "text"	d:3 MATCH
8	• <style>	d:3 MATCH
9	• <h1>	d:3 MATCH
10	• <p>	d:3 MATCH
11	• <p>	d:3 MATCH
12	• "text"	d:4 MATCH
13	• "text"	d:4 MATCH
14	• "text"	d:4 MATCH
15	• <a>	d:4 MATCH
16	• "text"	d:5 MATCH

- DFS

DOM Vector

Input a URL and an optional CSS selector to fetch and visualize its graphical structure.

Website URL

CSS Selector (Optional)

Top N Matches (0 = All)

Algorithm

☐ BFS
 ☒ DFS

☐ Multithreading
Subtree-parallel DFS

Playback Control

DOM Tree Topography

Analytics

LIVE

Matches Found **12 / 16**

Nodes Traversed **12 / 16**

Time Took **0 ms**

Max Depth **5**

Traversal Log

16 steps recorded

● MATCH ● Active ● VISIT

1	<html>	d:0	MATCH
2	<head>	d:1	MATCH
3	<title>	d:2	MATCH
4	"text"	d:3	MATCH
5	<meta>	d:2	MATCH
6	<style>	d:3	MATCH
7	"text"	d:4	MATCH
8	<body>	d:1	MATCH
9	<div>	d:2	MATCH
10	<h1>	d:3	MATCH
11	"text"	d:4	MATCH
12	<p>	d:3	MATCH

Analytics

Matches Found **16 / 16**

Nodes Traversed **16 / 16**

Time Took **0 ms**

Max Depth **5**

Traversal Log

16 steps recorded

● MATCH ● Active ● VISIT

5	<meta>	d:2	MATCH
6	<style>	d:3	MATCH
7	"text"	d:4	MATCH
8	<body>	d:1	MATCH
9	<div>	d:2	MATCH
10	<h1>	d:3	MATCH
11	"text"	d:4	MATCH
12	<p>	d:3	MATCH
13	"text"	d:4	MATCH
14	<p>	d:3	MATCH
15	<a>	d:4	MATCH
16	"text"	d:5	MATCH

- DFS dengan multithreading

DOM Vector

Input a URL and an optional CSS selector to fetch and visualize its graphical structure.

Website URL

CSS Selector (Optional)

Top N Matches (0 = All)

Algorithm

☐ BFS
 ☒ DFS

☒ Multithreading
 Subtree-parallel DFS

Playback Control

DOM Tree Topography

Analytics LIVE

Matches Found **13 / 16**

Nodes Traversed **13 / 16**

Time Took **0 ms**

Max Depth **5**

Traversal Log 16 steps recorded

☒ MATCH
 ☒ Active
 ☐ VISIT

	SELECT	U+U	PRELIM
2	<head>	d:1	MATCH
3	<title>	d:2	MATCH
4	"text"	d:3	MATCH
5	<meta>	d:2	MATCH
6	<style>	d:3	MATCH
7	"text"	d:4	MATCH
8	<body>	d:1	MATCH
9	<div>	d:2	MATCH
10	<h1>	d:3	MATCH
11	"text"	d:4	MATCH
12	<p>	d:3	MATCH
13	"text"	d:4	MATCH

Analytics

Matches Found **16 / 16**

Nodes Traversed **16 / 16**

Time Took **0 ms**

Max Depth **5**

Traversal Log 16 steps recorded

☒ MATCH
 ☒ Active
 ☐ VISIT

	SELECT	U+U	PRELIM
5	<meta>	d:2	MATCH
6	<style>	d:3	MATCH
7	"text"	d:4	MATCH
8	<body>	d:1	MATCH
9	<div>	d:2	MATCH
10	<h1>	d:3	MATCH
11	"text"	d:4	MATCH
12	<p>	d:3	MATCH
13	"text"	d:4	MATCH
14	<p>	d:3	MATCH
15	<a>	d:4	MATCH
16	"text"	d:5	MATCH

4.2.2. Test Case 2

Input: [https://en.wikipedia.org/wiki/Constantine_\(son_of_Theophilos\)](https://en.wikipedia.org/wiki/Constantine_(son_of_Theophilos))

Output:

- DFS dengan CSS Selector: footer#footer

DOM Vector
Input a URL and an optional CSS selector to fetch and visualize its graphical structure.

URL: Raw HTML

Website URL:

CSS Selector (Optional):

Top N Matches (0 = All):

Algorithm: ☒ BFS ☒ DFS

☐ Multithreading
Subtree-parallel DFS

Playback Control:

DOM Tree Topography
Tree terlalu besar! Menampilkan 300 node relevan dari 2657 total

Analytics LIVE

Matches Found: 0 / 1

Nodes Traversed: 18 / 2657

Time Took: 2 ms

Max Depth: 26

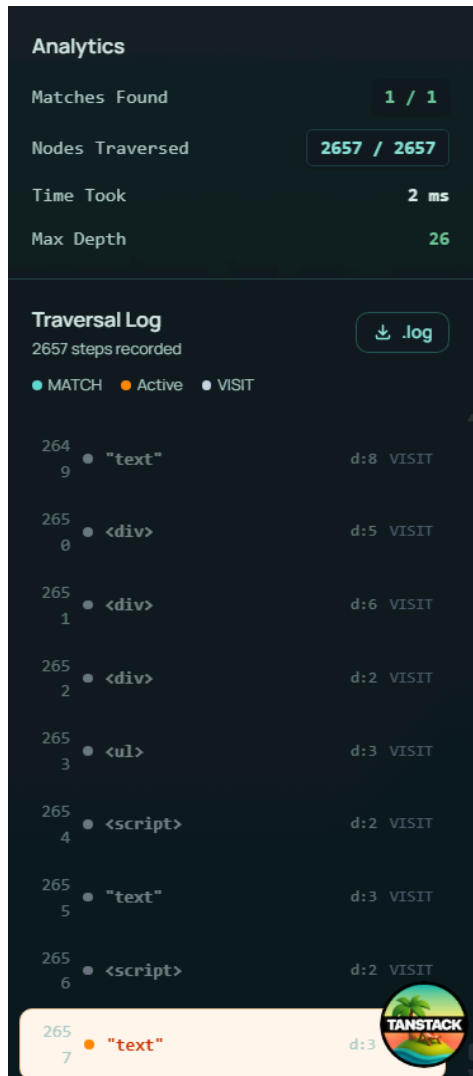
Traversal Log

2657 steps recorded

● MATCH ● Active ● VISIT

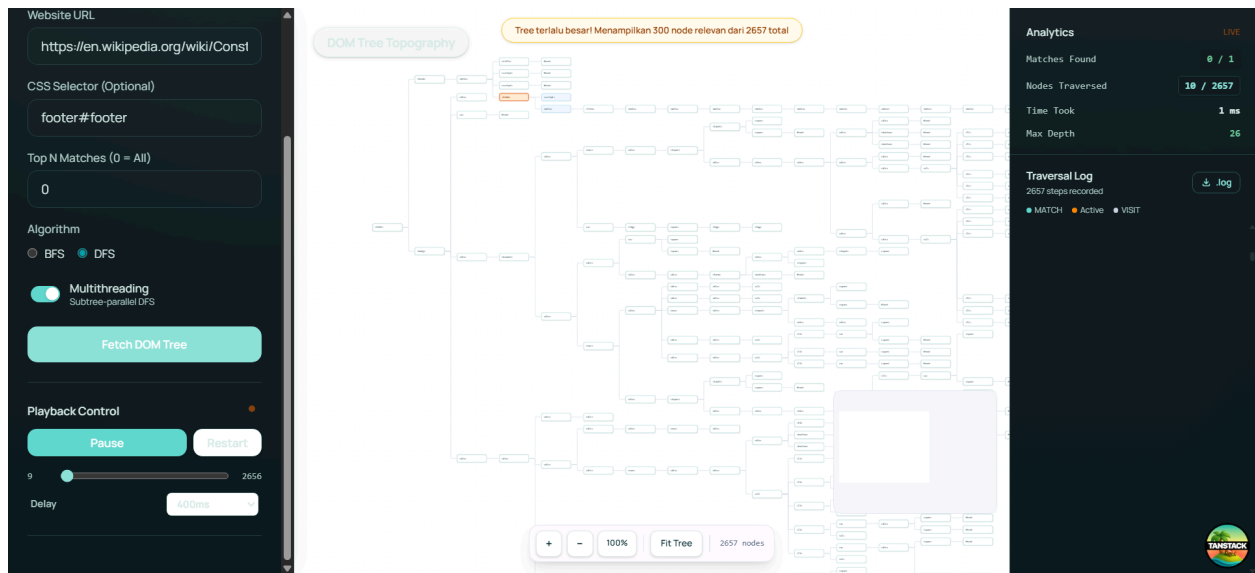
5	● "text"	d:4 VISIT
6	● <script>	d:3 VISIT
7	● "text"	d:4 VISIT
8	● <script>	d:3 VISIT
9	● "text"	d:4 VISIT
10	● <link>	d:3 VISIT
11	● <script>	d:4 VISIT
12	● <meta>	d:4 VISIT
13	● <link>	d:5 VISIT
14	● <meta>	d:6 VISIT
15	● <meta>	d:7 VISIT
16	● <meta>	d:8 VISIT
17	● <meta>	d:9 VISIT
18	● <meta>	d:10 VISIT

TANGTACK



- DFS dengan multithreading dan CSS Selector: footer#footer

IF2211 Strategi Algoritma - Tugas Besar 2



Analytics

Matches Found

1 / 1

Nodes Traversed

2657 / 2657

Time Took

1 ms

Max Depth

26

Traversal Log

2657 steps recorded

● MATCH

● Active

● VISIT

264

9

● "text"

d:8 VISIT

265

0

● <div>

d:5 VISIT

265

1

● <div>

d:6 VISIT

265

2

● <div>

d:2 VISIT

265

3

●

d:3 VISIT

265

4

● <script>

d:2 VISIT

265

5

● "text"

d:3 VISIT

265

6

● <script>

d:2 VISIT

265

7

● "text"

d:3

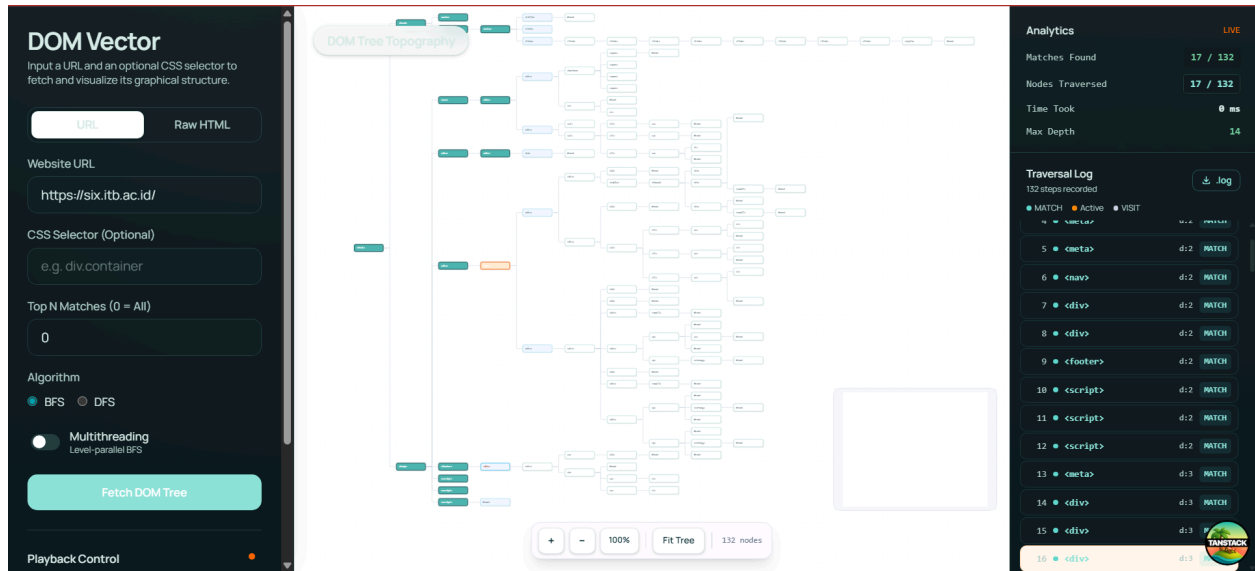
TANSTACK

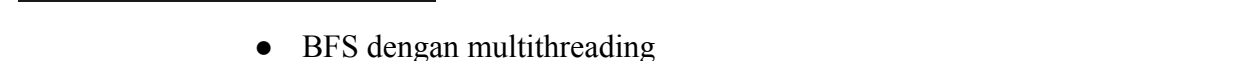
4.2.3. Test Case 3

Input: <https://six.itb.ac.id/>

Output:

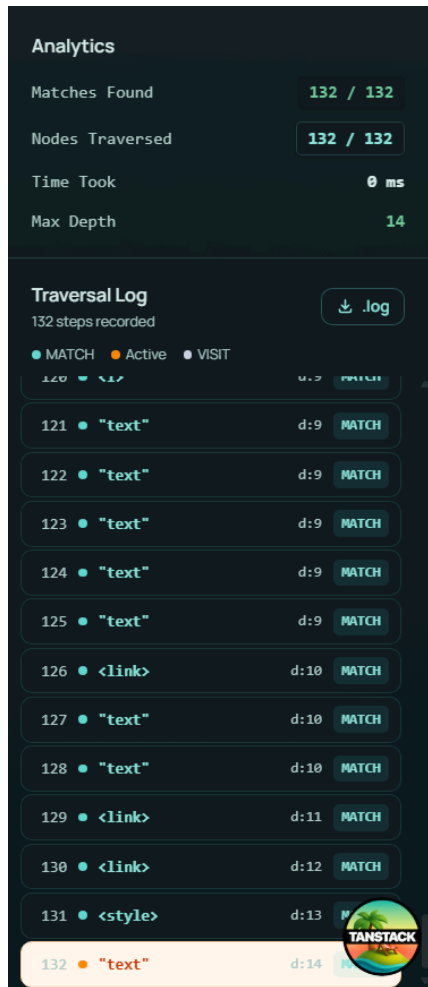
- BFS





Website URL





4.2.4. Test Case 4

Input:

```
<!DOCTYPE html>
<html>
<head>
  <title>Stress Test DOM Traversal</title>
</head>
<body>
  <div id="level-1" class="wrapper">
    <div id="level-2" class="wrapper">
      <div id="level-3" class="wrapper">
        <div id="level-4" class="wrapper">
          <div id="level-5" class="wrapper">
            <div id="level-6" class="wrapper">
              <div id="level-7" class="wrapper">
                <div id="level-8"
class="wrapper">
                  <div id="level-9"
```

```
class="wrapper">
    <div id="level-10"
class="wrapper">
    <p
class="target-ekstrem">Selamat! Eksekusi IDDFS dan Parser
berhasil menembus kedalaman 10 level.</p>
    </div>
    </div>
    </div>
    </div>
    </div>
    </div>
    </div>
    </div>
    </div>
    </div>
    <div class="pengecoh">
    <p class="target-ekstrem">Ini node pengecoh yang
letaknya dangkal, BFS harusnya nemu ini duluan.</p>
    </div>
</body>
</html>
```

Output:

- BFS dengan CSS Selector: p.target-ekstrem

Raw HTML

URL

Raw HTML

DOM Tree Topography

Analytics

Matches Found: 1 / 2

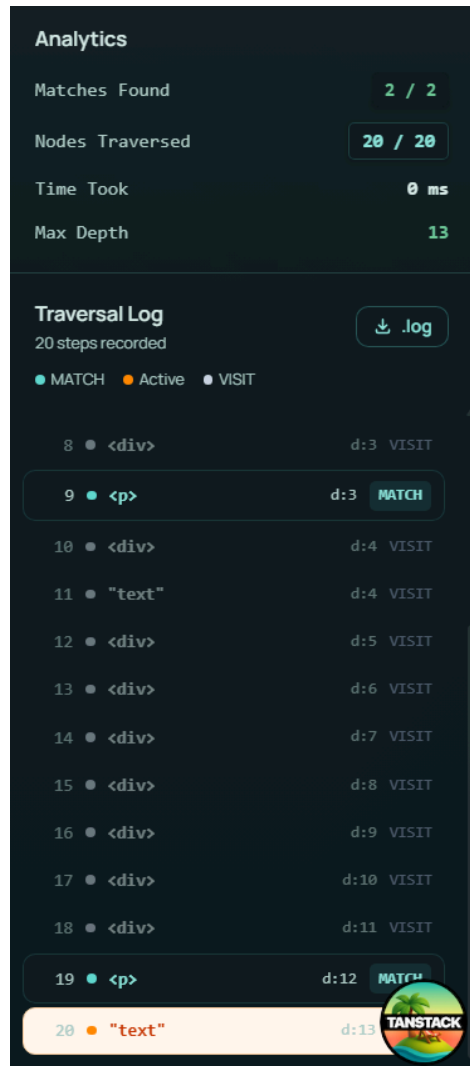
Nodes Traversed: 10 / 20

Time Took: 0 ms

Max Depth: 13

Traversal Log

Step	Node	Type	Depth	Status
1	<html>	Root	d:0	VISIT
2	<head>	Child	d:1	VISIT
3	<body>	Child	d:1	VISIT
4	<title>	Child	d:2	VISIT
5	<div>	Child	d:2	VISIT
6	<div>	Child	d:2	VISIT
7	"text"	Text	d:3	VISIT
8	<div>	Child	d:3	VISIT
9	<p>	Child	d:3	MATCH
10	<div>	Child	d:4	VISIT
11	"text"	Text	d:4	VISIT
12	<div>	Child	d:5	VISIT
13	<div>	Child	d:5	VISIT
14	<div>	Child	d:7	VISIT



- DFS dengan multithreading dan CSS Selector: p.target-ekstrem

URL

Raw HTML

Raw HTML

```

<div class= pengecon >
  <p class="target-ekstrem">Ini node pengecoh yang letaknya dangkal, BFS harusnya nemu ini duluan.</p>
</div>
</body>
</html>

```

CSS Selector (Optional)

p.target-ekstrem

Top N Matches (0 = All)

0

Algorithm

☐ BFS
 ☒ DFS

☒ Multithreading
Subtree-parallel DFS

Parse HTML

Playback Control

DOM Tree Topography

+

-

100%

Fit Tree

28 nodes

Analytics

LIVE

Matches Found

0 / 2

Nodes Traversed

6 / 28

Time Took

0 ms

Max Depth

13

Traversal Log

20 steps recorded

☒ MATCH
 ☒ Active
 ☐ VISIT

1	•	<html>	d:0	VISIT
2	•	<head>	d:1	VISIT
3	•	<title>	d:2	VISIT
4	•	"text"	d:3	VISIT
5	•	<body>	d:1	VISIT
6	•	<div>	d:2	VISIT
7	•	<div>	d:3	VISIT
8	•	<div>	d:4	VISIT
9	•	<div>	d:5	VISIT
10	•	<div>	d:6	VISIT
11	•	<div>	d:7	VISIT
12	•	<div>	d:8	VISIT
13	•	<div>	d:9	VISIT
14	•	<div>	d:10	VISIT

Analytics

Matches Found

2 / 2

Nodes Traversed

20 / 28

Time Took

0 ms

Max Depth

13

Traversal Log

20 steps recorded

☒ MATCH
 ☒ Active
 ☐ VISIT

8	•	<div>	d:4	VISIT
9	•	<div>	d:5	VISIT
10	•	<div>	d:6	VISIT
11	•	<div>	d:7	VISIT
12	•	<div>	d:8	VISIT
13	•	<div>	d:9	VISIT
14	•	<div>	d:10	VISIT
15	•	<div>	d:11	VISIT
16	•	<p>	d:12	MATCH
17	•	"text"	d:13	VISIT
18	•	<div>	d:2	VISIT
19	•	<p>	d:3	MATCH
20	•	"text"	d:4	

4.2.5. Test Case 5

Input: <https://inibukanurl.com>

Output:

4.3. Analisis Pengujian

Berdasarkan hasil eksekusi dari berbagai skenario uji coba (mulai dari laman web sederhana, struktur bersarang ekstrem, hingga struktur laman portal modern), program secara konsisten berhasil memodelkan teks HTML mentah menjadi pohon DOM dan mengevaluasi selektor CSS secara akurat. Dari data pengujian tersebut, dapat ditarik beberapa poin analisis perbandingan antara algoritma Breadth-First Search (BFS) dan Depth-First Search (DFS):

- Pengujian membuktikan bahwa lokasi elemen pada struktur pohon DOM sangat memengaruhi waktu eksekusi masing-masing algoritma. Pada skenario web modern di mana elemen target penyusun kerangka (seperti header, nav, atau

container utama) terletak pada kedalaman yang dangkal, algoritma BFS unggul secara signifikan. Hal ini dikarenakan sifat level-order traversal yang memungkinkan BFS langsung menemukan target tanpa harus menelusuri simpul anak yang tidak relevan. Sebaliknya, DFS menunjukkan keunggulannya pada skenario pencarian elemen yang terletak sangat dalam pada cabang spesifik (seperti daftar referensi di bagian footer laman yang panjang).

- Pencatatan metrik traversal log mengonfirmasi teori kompleksitas ruang dari kedua algoritma. Pada laman web dengan struktur yang sangat lebar (banyak elemen sibling di satu level), ukuran antrean (queue) pada BFS membengkak cukup besar. Di sisi lain, IDDFS menjaga penggunaan memori tetap konstan dan sangat efisien karena beroperasi menggunakan stack yang dibatasi kedalamannya. Namun, efisiensi memori pada IDDFS harus dibayar dengan kompromi pada sisi waktu; algoritma ini sering kali mengunjungi node yang sama berulang kali di level atas setiap kali batas kedalaman (depth limit) diinkrementasi.
- Pengujian menggunakan selektor majemuk dan kombinator (seperti p.class + div) membuktikan bahwa modul selector berhasil dieksekusi dengan sempurna sebagai goal test pada setiap iterasi BFS maupun IDDFS tanpa membebani alur traversal utama. Selain itu, penggunaan IDDFS terbukti berhasil mencegah terjadinya risiko infinite loop atau stack overflow yang biasa terjadi pada DFS murni ketika dihadapkan pada graf berukuran masif.

BAB 5

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Tugas Besar 2 ini berhasil mengimplementasikan mekanisme penelusuran CSS pada pohon Document Object Model menggunakan pendekatan algoritma BFS dan DFS. Dari hasil implementasi dan pengujian, dapat ditarik beberapa kesimpulan sebagai berikut.

1. Program telah berhasil memodelkan teks HTML menjadi struktur data pohon (DOM Tree) dan mengimplementasikan algoritma Breadth-First Search (BFS) serta Iterative Deepening Depth-First Search (IDDFS) untuk melakukan penelusuran elemen berdasarkan input CSS Selector dari pengguna.
2. Berdasarkan pengujian performa, algoritma BFS merupakan pilihan yang paling optimal dan direkomendasikan untuk menelusuri halaman website pada umumnya. Hal ini karena elemen-elemen paling krusial pada struktur HTML biasanya terletak pada kedalaman yang dangkal, sehingga BFS menjamin rute penemuan terpendek (shortest path).
3. Implementasi IDDFS merupakan alternatif yang sangat efisien dalam hal penggunaan memori dibandingkan BFS pada pohon yang teramat lebar, sekaligus jauh lebih aman dibandingkan DFS konvensional karena memiliki batasan kedalaman terukur yang mencegah pencarian tanpa akhir.
4. Arsitektur client-server yang didukung dengan orkestrasi kontainer Docker dan deployment melalui Azure Virtual Machine terbukti memberikan lingkungan eksekusi yang tangguh, konsisten, dan dapat diakses secara publik.

5.2 Saran

Berdasarkan hasil implementasi dan pengujian, beberapa saran pengembangan yang dapat dilakukan untuk pengembangan dan penyempurnaan sistem penelusuran lebih lanjut di masa mendatang adalah sebagai berikut.

1. Saat ini, modul scraper hanya mengambil teks HTML statis. Sistem dapat dikembangkan lebih lanjut dengan mengintegrasikan headless browser (seperti Puppeteer atau Selenium) agar mampu merender dan menelusuri elemen-elemen

DOM yang dihasilkan secara dinamis oleh JavaScript pada aplikasi berbasis Client-Side Rendering (CSR).

2. Modul parser selektor dapat diperluas untuk mendukung selektor pseudo-class tingkat lanjut pada CSS, seperti :nth-child(), :first-of-type, atau :empty, guna memberikan fleksibilitas pencarian yang lebih tinggi bagi pengguna.
3. Penerapan paralelisasi (multithreading) dapat dioptimalkan lebih jauh, tidak hanya dengan membagi pekerjaan pada tingkat node, tetapi dengan mengimplementasikan mekanisme worker pool dinamis untuk meminimalkan overhead pembuatan goroutine pada pohon DOM berukuran kecil

Diharapkan dengan evaluasi dan saran ini, dapat meningkatkan kualitas sistem penelusuran yang lebih baik.

LAMPIRAN

Link Terkait

Link Repositori GitHub	https://github.com/nicholaswisee/Tubes2_TimsesDewaPetir
Link Deployment	http://20.6.132.22/

Tabel Pembagian Tugas

Nama Lengkap	NIM	Tugas
Nicholas Wise Saragih Sumbayak	13524037	● Implementasi backend ● Implementasi frontend ● Implementasi bonus ● Laporan
Narendra Dharma Wistara M	13524044	● Implementasi frontend ● Implementasi bonus ● Laporan
Amanda Aurellia Salsabilla	13524044	● Implementasi backend ● Pengujian ● Laporan

Tabel Checklist

No	Poin	Ya	Tidak
1	Aplikasi berhasil di kompilasi tanpa kesalahan	✓	
2	Aplikasi berhasil dijalankan	✓	
3	Aplikasi dapat menerima input URL web, pilihan algoritma, CSS selector, dan jumlah hasil	✓	
4	Aplikasi dapat melakukan scraping terhadap web pada input	✓	
5	Aplikasi dapat menampilkan visualisasi pohon DOM	✓	
6	Aplikasi dapat menelusuri pohon DOM dan menampilkan hasil penelusuran	✓	
7	Aplikasi dapat menandai jalur tempuh oleh algoritma	✓	
8	Aplikasi dapat menyimpan jalur yang ditempuh algoritma dalam traversal log	✓	

9	[Bonus] Membuat video		✓
10	[Bonus] Deploy aplikasi	✓	
11	[Bonus] Implementasi animasi pada penelusuran pohon	✓	
12	[Bonus] Implementasi multithreading	✓	
13	[Bonus] Implementasi LCA Binary Lifting	✓	

DAFTAR PUSTAKA

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
- CP-Algorithms, "Lowest Common Ancestor - Binary Lifting," *CP-Algorithms*. [Online]. Available: https://cp-algorithms.com/graph/lca_binary_lifting.html. [Diakses: 23-Apr-2026].
- GeeksforGeeks, "Depth First Search or DFS for a Graph," *GeeksforGeeks*. [Online]. Available: <https://www.geeksforgeeks.org/dsa/depth-first-search-or-dfs-for-a-graph/>. [Diakses: 23-Apr-2026].
- GeeksforGeeks, "Breadth First Search or BFS for a Graph," *GeeksforGeeks*. [Online]. Available: <https://www.geeksforgeeks.org/dsa/breadth-first-search-or-bfs-for-a-graph/>. [Diakses: 23-Apr-2026].
- Laboratorium Ilmu dan Rekayasa Komputasi. (2026). *Spesifikasi Tugas Besar 2 IF2211 Strategi Algoritma: Pemanfaatan Algoritma BFS dan DFS dalam Mekanisme Penelusuran CSS pada pohon Document Object Model*. Institut Teknologi Bandung.
- MDN Web Docs, "Document Object Model (DOM)," *Mozilla*. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model/html_dom. [Diakses: 23-Apr-2026].
- MDN Web Docs, "HTML: HyperText Markup Language," *Mozilla*. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/HTML>. [Diakses: 23-Apr-2026].
- MDN Web Docs, "CSS selectors," *Mozilla*. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Selectors>. [Diakses: 23-Apr-2026].
- Munir, R. (2026). *Materi Kuliah IF2211 Strategi Algoritma - Breadth First Search (BFS) dan Depth First Search (DFS) Bagian 1*. Program Studi Teknik Informatika, Institut Teknologi Bandung.
- Munir, R. (2026). *Materi Kuliah IF2211 Strategi Algoritma - Breadth First Search (BFS) dan Depth First Search (DFS) Bagian 2*. Program Studi Teknik Informatika, Institut Teknologi Bandung.
- Russell, S., & Norvig, P. (2010). *Artificial Intelligence: A Modern Approach* (3rd ed.). Prentice Hall.